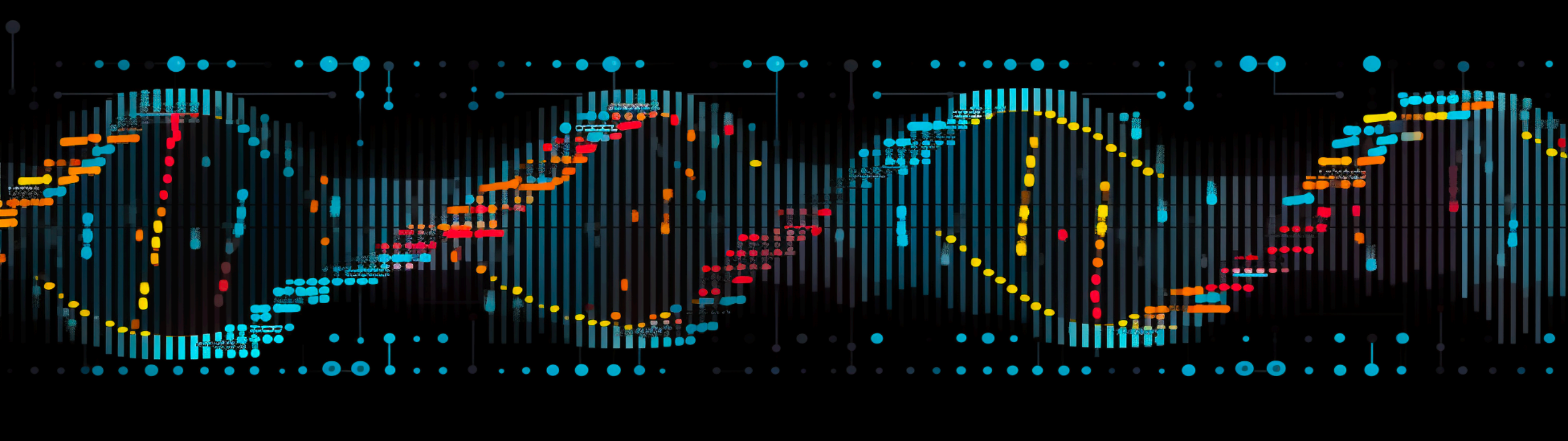




Introduction to bioinformatics

Chapter 4 Machine learning in bioinformatics

- Matrix factorization, imputation and dimension reduction

Jiaxing Chen

Outline

- **Matrix factorization**
 - SVD & NMF
- NMF in bioinformatics
- Imputation
 - Proteomics
 - Single cell RNA-seq
- Dimension reduction
 - Overview
 - Isomap, t-SNE and UMAP
 - 3D reconstruction from single cell RNA-seq

Non-Negative Datasets

Some datasets are intrinsically non-negative:

Counters (e.g., no. occurrences of each word in a text document)

Quantities (e.g., amount of each ingredient in a chemical experiment)

Intensities (e.g., intensity of each color in an image)

The corresponding data matrix D has only non-negative values. Decompositions such as SVD and SDD may involve negative values in factors and components
Negative values describe the absence of something Often no natural interpretation

Can we find a decomposition that is more natural to non-negative data?

Example (SVD)

Consider the following “bridge” matrix and its truncated SVD:

$$\begin{array}{ccccc}
 \begin{array}{|c|c|c|c|c|}
 \hline
 1 & 1 & 1 & 1 & 1 \\
 \hline
 0 & 1 & 0 & 1 & 0 \\
 \hline
 0 & 1 & 0 & 1 & 0 \\
 \hline
 \end{array} & = & \begin{array}{|c|c|}
 \hline
 0.8 & 0.6 \\
 \hline
 0.5 & -0.5 \\
 \hline
 0.5 & -0.5 \\
 \hline
 \end{array} & \begin{array}{|c|c|}
 \hline
 1.5 & 0 \\
 \hline
 0 & 1.3 \\
 \hline
 \end{array} & \begin{array}{|c|c|c|c|c|}
 \hline
 0.3 & 0.6 & 0.3 & 0.6 & 0.3 \\
 \hline
 0.5 & -0.3 & 0.5 & -0.3 & 0.5 \\
 \hline
 \end{array} \\
 \mathbf{D} & & \mathbf{U} & \mathbf{\Sigma} & \mathbf{V}^T
 \end{array}$$

Here are the corresponding components:

$$\begin{array}{ccccc}
 \begin{array}{|c|c|c|c|c|}
 \hline
 1 & 1 & 1 & 1 & 1 \\
 \hline
 0 & 1 & 0 & 1 & 0 \\
 \hline
 0 & 1 & 0 & 1 & 0 \\
 \hline
 \end{array} & = & \begin{array}{|c|c|c|c|c|}
 \hline
 0.6 & 1.3 & 0.6 & 1.3 & 0.6 \\
 \hline
 0.3 & 0.8 & 0.3 & 0.8 & 0.3 \\
 \hline
 0.3 & 0.8 & 0.3 & 0.8 & 0.3 \\
 \hline
 \end{array} & + & \begin{array}{|c|c|c|c|c|}
 \hline
 0.4 & -0.3 & 0.4 & -0.3 & 0.4 \\
 \hline
 -0.3 & 0.2 & -0.3 & 0.2 & -0.3 \\
 \hline
 -0.3 & 0.2 & -0.3 & 0.2 & -0.3 \\
 \hline
 \end{array} \\
 \mathbf{D} & & \mathbf{U}_{*1} \mathbf{D}_{11} \mathbf{V}_{*1}^T & & \mathbf{U}_{*2} \mathbf{D}_{22} \mathbf{V}_{*2}^T
 \end{array}$$

Negative values make interpretation unnatural or difficult.

Non-Negative Matrix Factorization (NMF)

Definition (Non-negative matrix factorization, basic form)

Given a non-negative matrix $\mathbf{D} \in \mathbb{R}_+^{m \times n}$, a *non-negative matrix factorization* of rank k is

$$\mathbf{D} \approx \mathbf{L}\mathbf{R},$$

where $\mathbf{L} \in \mathbb{R}_+^{m \times r}$ and $\mathbf{R} \in \mathbb{R}_+^{r \times n}$ are both non-negative.

- Additive decomposition: factors and components non-negative
→ No cancellation effects
- Rows of $\mathbf{R}$ can be thought as “parts”
- Row of $\mathbf{D}$ obtained by mixing (or “assembling”) parts in $\mathbf{L}$
- Smallest r such that $\mathbf{D} = \mathbf{L}\mathbf{R}$ exists is called *non-negative rank* of $\mathbf{D}$

$$\text{rank}(\mathbf{D}) \leq \text{rank}_+(\mathbf{D}) \leq \min \{ m, n \}$$

Example (NMF)

Consider the following “bridge” matrix and its rank-2 NMF:

$$\mathbf{D} = \mathbf{L} \mathbf{R}$$

1	1	1	1	1
0	1	0	1	0
0	1	0	1	0

1	0
0	1
0	1

1	1	1	1	1
0	1	0	1	0

Here are the corresponding components:

$$\mathbf{D} = \mathbf{L}_{*1} \mathbf{R}_{1*} + \mathbf{L}_{*2} \mathbf{R}_{2*}$$

1	1	1	1	1
0	1	0	1	0
0	1	0	1	0

1	1	1	1	1
0	0	0	0	0
0	0	0	0	0

0	0	0	0	0
0	1	0	1	0
0	1	0	1	0

Non-negative matrix decomposition encourage a more natural, part-based representation and (sometimes) sparsity.

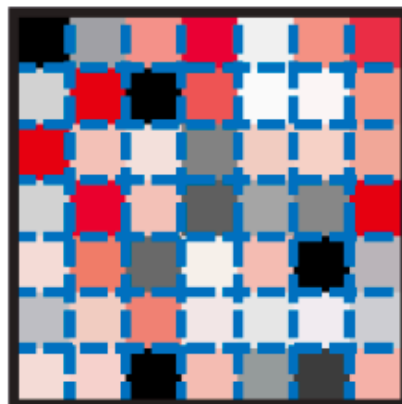
Decomposing faces (PCA)



D_{i*} (original)



$[LR]_{i*}$
 $[U\Sigma V^T]_{i*}$



L_{i*}
 $U_{i*}\Sigma$



R
 V^T

PCA factors are hard to interpret.

Decomposing faces (NMF)

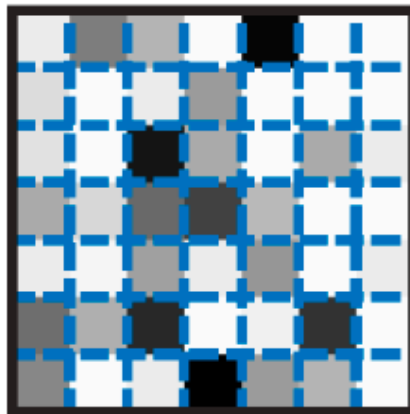


D_{i*} (original)

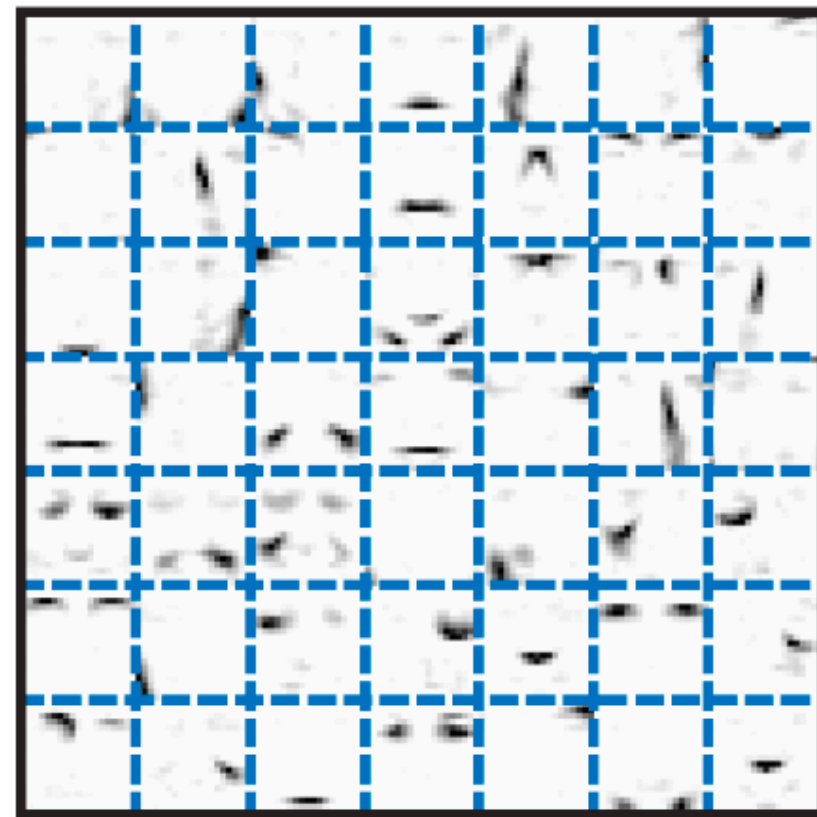


$[LR]_{i*}$

=



L_{i*}



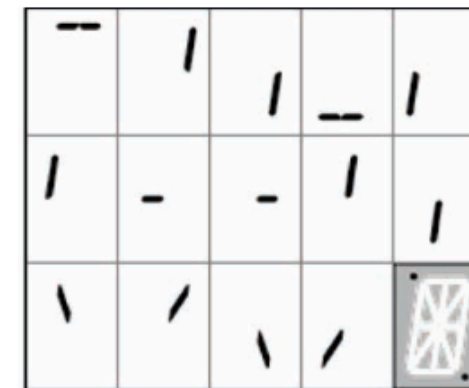
R

NMF factors correspond to parts of faces.

Decomposing digits (NMF)



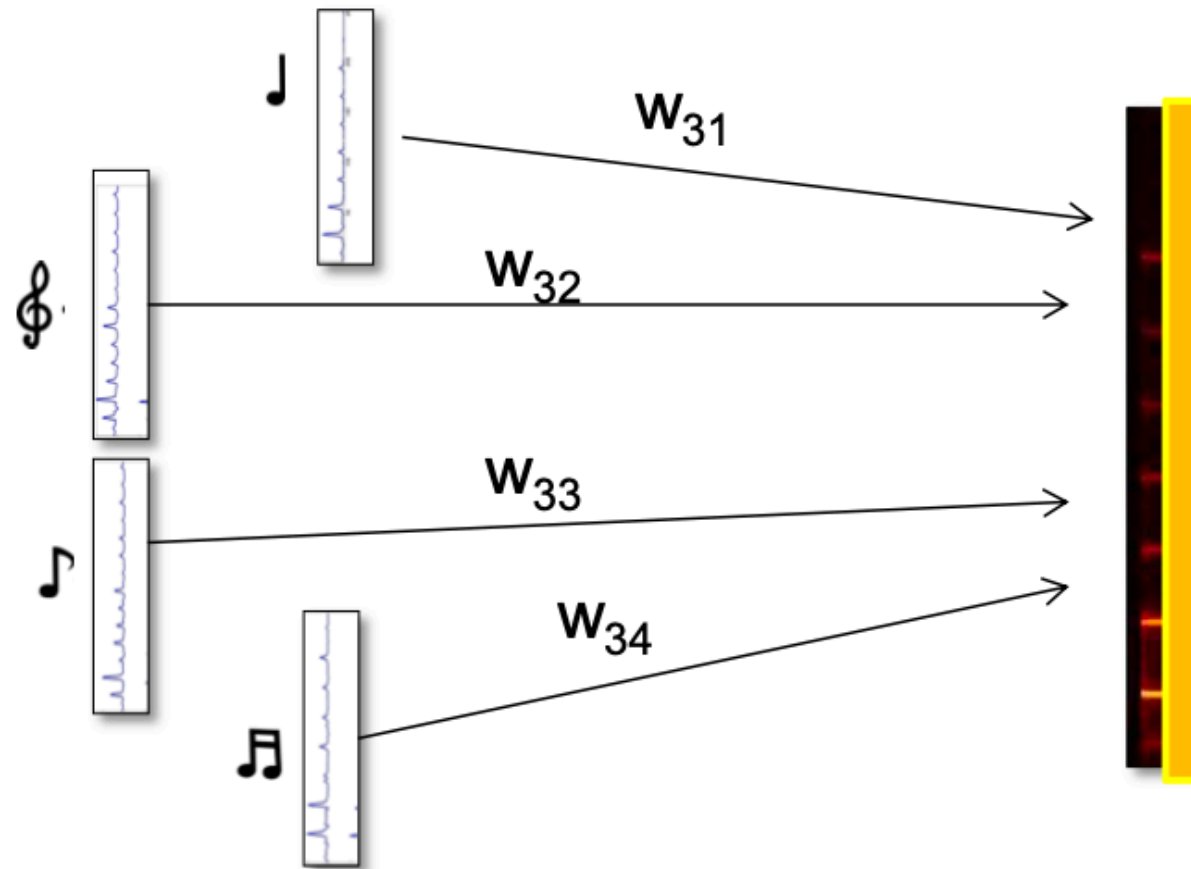
D



R

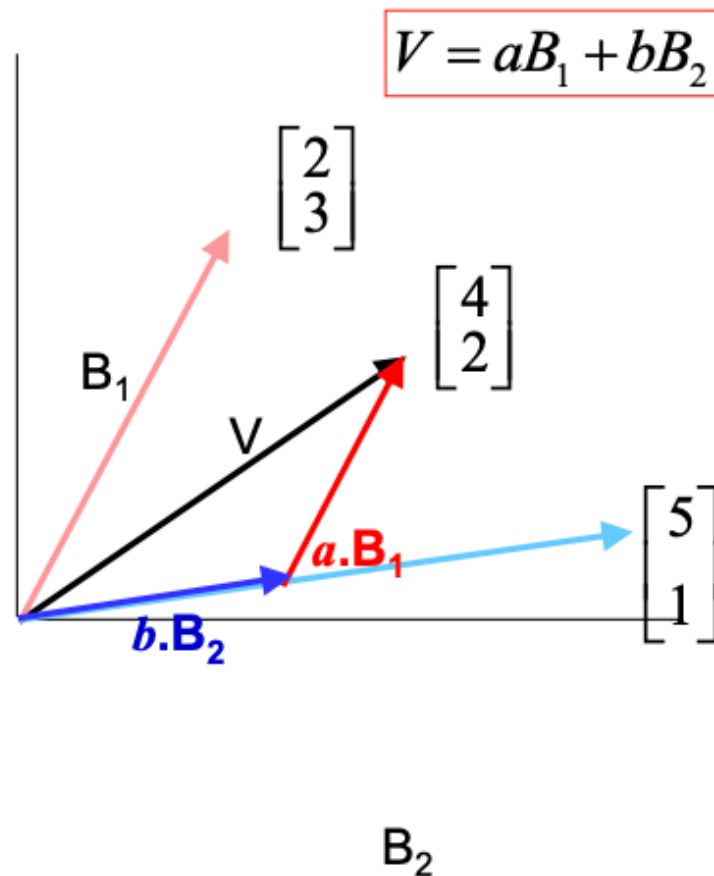
NMF factors correspond to parts of digits and “background” .

Building Blocks of Sound



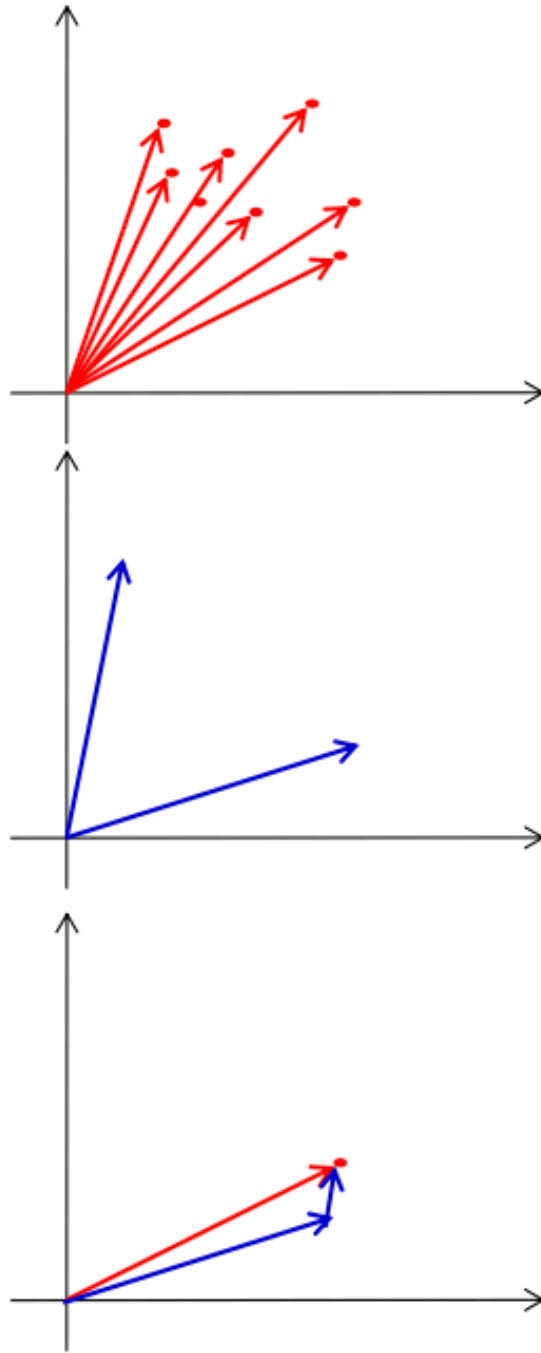
- Each frame of sound is composed by activating each spectral building block by a frame-specific amount
- Individual frames are composed by activating the building blocks to different degrees
 - E.g. notes are strummed with different energies to compose the frame

Expressing a vector in terms of other vectors

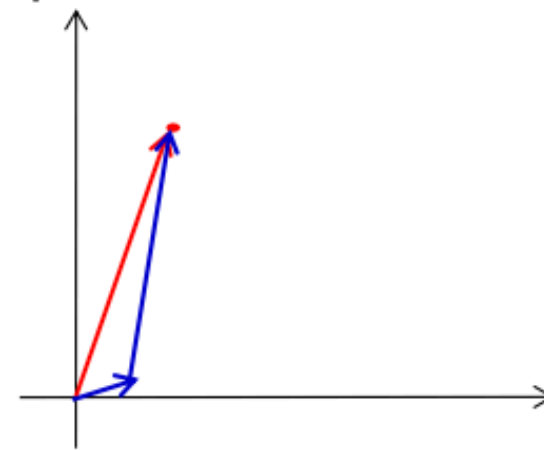
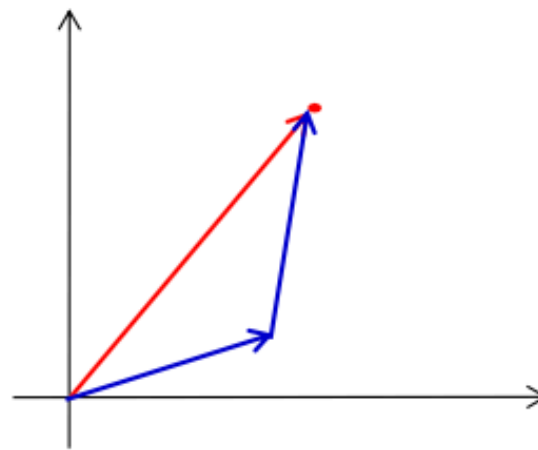


- V has only non-negative components
 - Is a power spectrum
- B_1 and B_2 have only non-negative components
 - Power spectra of building blocks of audio
 - E.g. power spectra of notes
- a and b are strictly non-negative
 - Building blocks don't subtract from one another

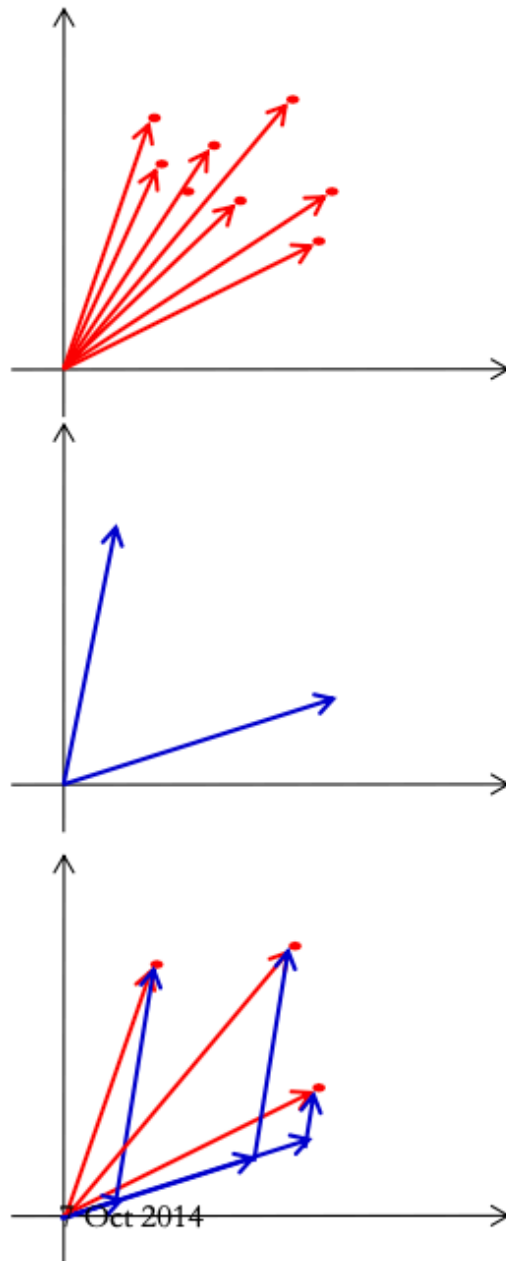
Using power spectral vector (composed sound) decomposition as example



- Given a collection of spectral vectors (from the composed sound) ...
- Find a set of “basic” sound spectral vectors such that ...
- All of the spectral vectors can be composed through constructive addition of the bases
 - We never have to flip the direction of any basis



Restating the problem



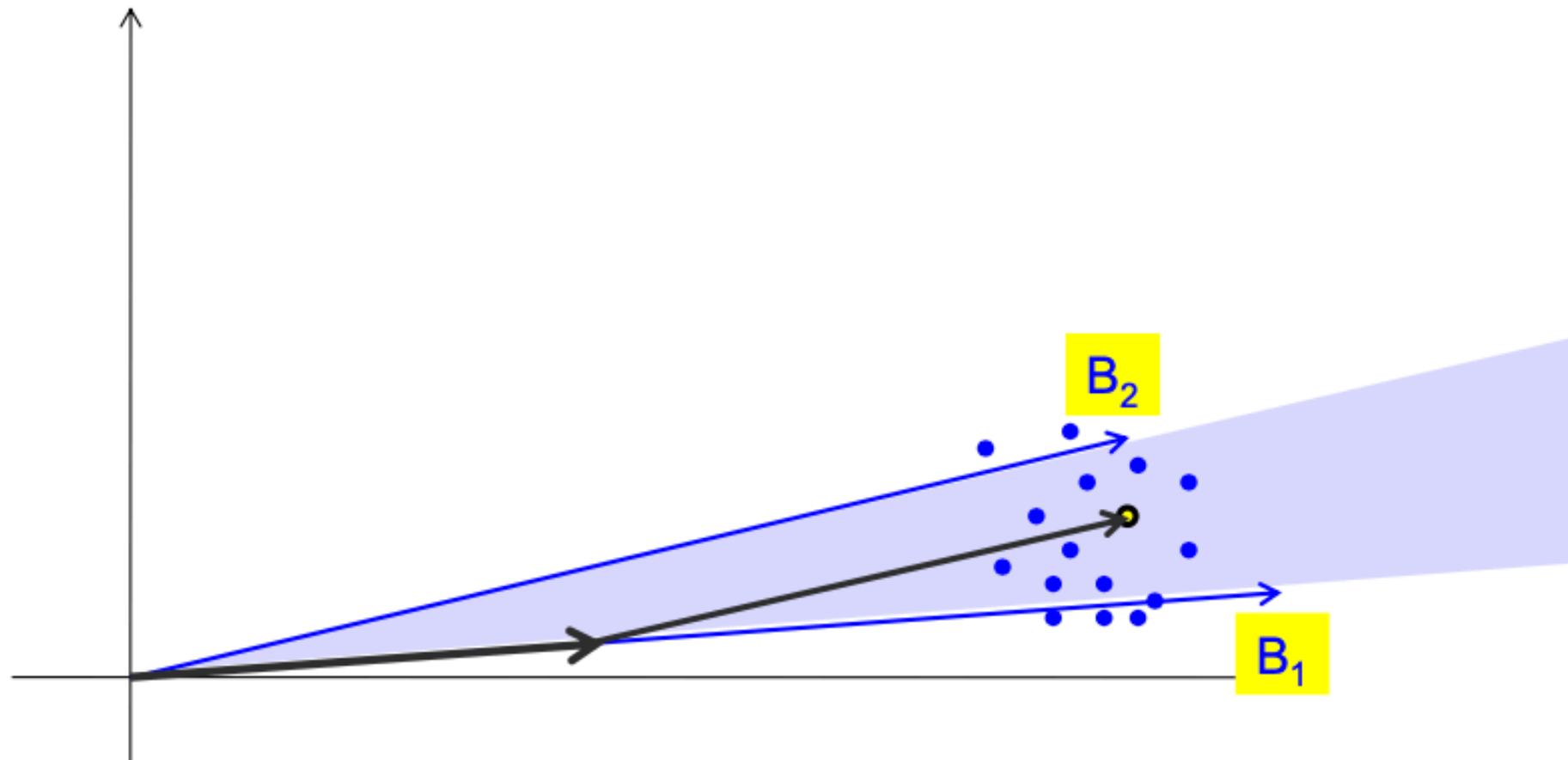
$$\mathbf{V} = \mathbf{B}\mathbf{W}$$

- Each column of $\mathbf{V}$ is one “composed” spectral vector
- Each column of $\mathbf{B}$ is one building block
 - One spectral basis
- Each column of $\mathbf{W}$ has the scaling factors for the building blocks to compose the corresponding column of $\mathbf{V}$
- All columns of $\mathbf{V}$ are non-negative
- All entries of $\mathbf{B}$ and $\mathbf{W}$ must also be non-negative

Non-negative matrix factorization: Basics

- NMF is used in a *compositional* model
- Data are assumed to be non-negative
 - E.g. power spectra
- Every data vector is explained as a purely constructive linear composition of a set of bases
 - $V = \sum_i w_i B_i$
 - The bases B_i are in the same domain as the data
 - I.e. they are power spectra
- Constructive composition: no subtraction allowed
 - Weights w_i must all be non-negative
 - All components of bases B_i must also be non-negative

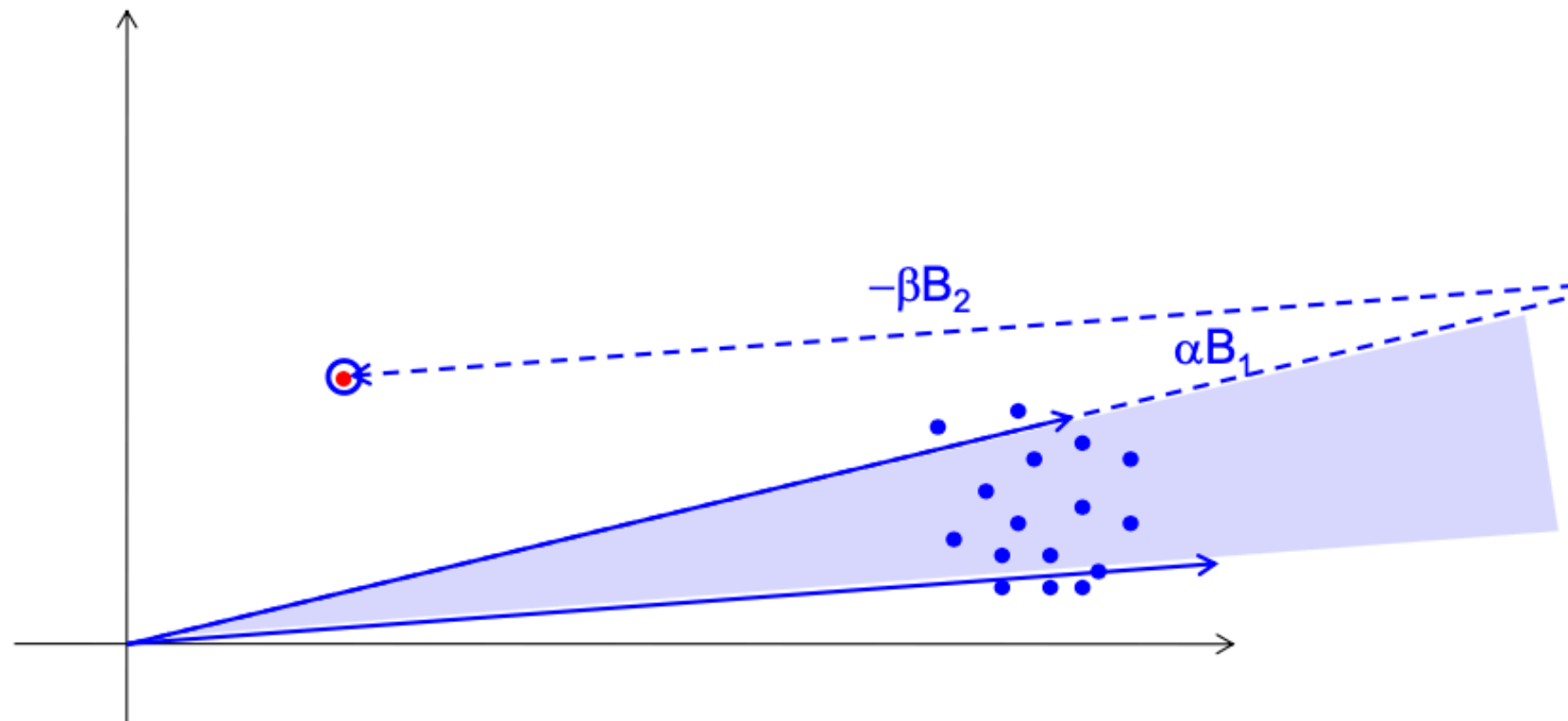
Interpreting non-negative factorization



- Bases are non-negative, lie in the positive quadrant
- Blue lines represent bases, blue dots represent vectors
- Any vector that lies between the bases (highlighted region) can be expressed as a non-negative combination of bases

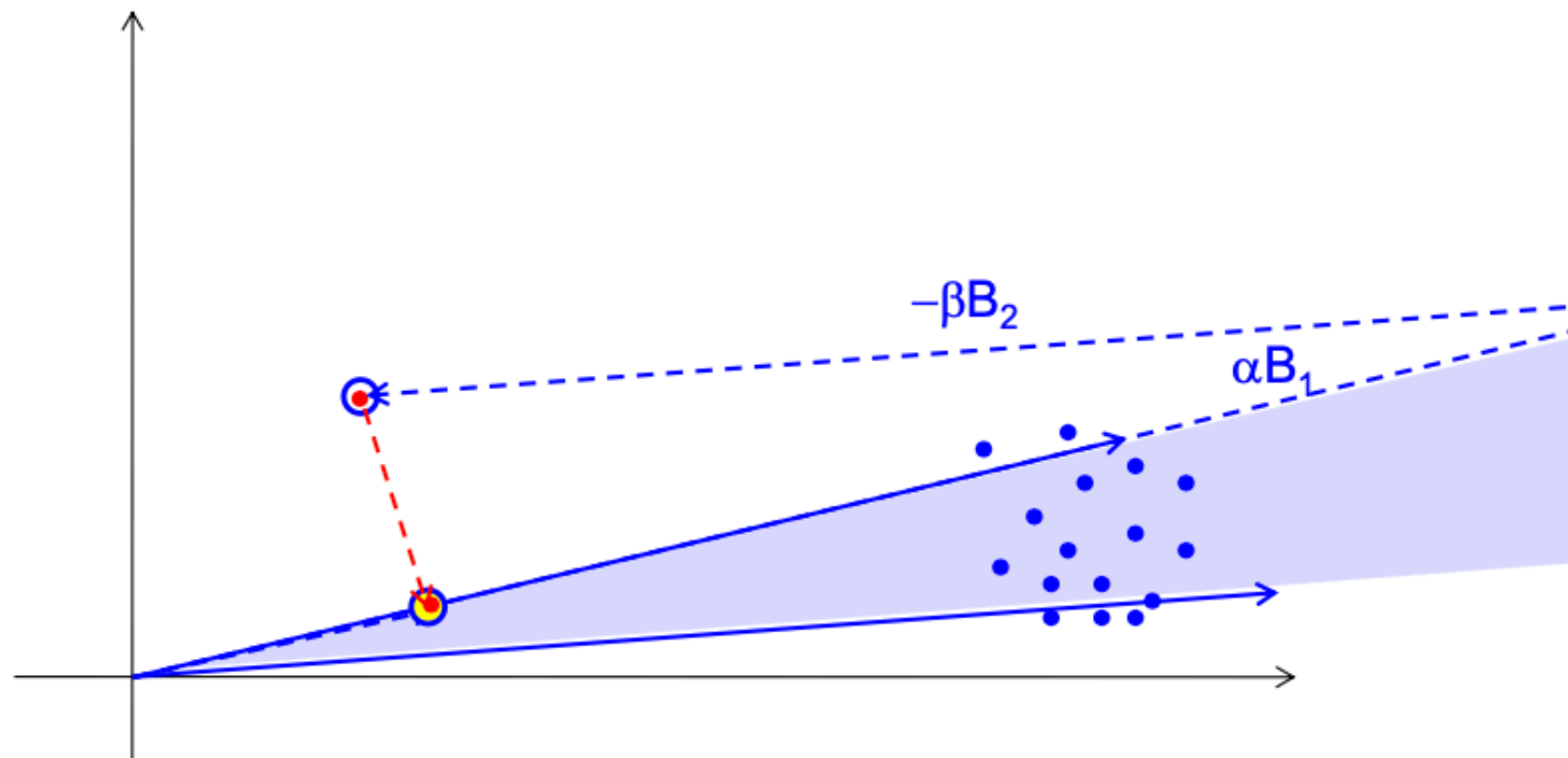
E.g. the black dot

Interpreting non-negative factorization



- Vectors outside the shaded enclosed area can only be expressed as a linear combination of the bases by reversing a basis
 - I.e. assigning a negative weight to the basis
 - E.g. the red dot
 - Alpha and beta are scaling factors for bases
 - Beta weighting is negative

Interpreting non-negative factorization

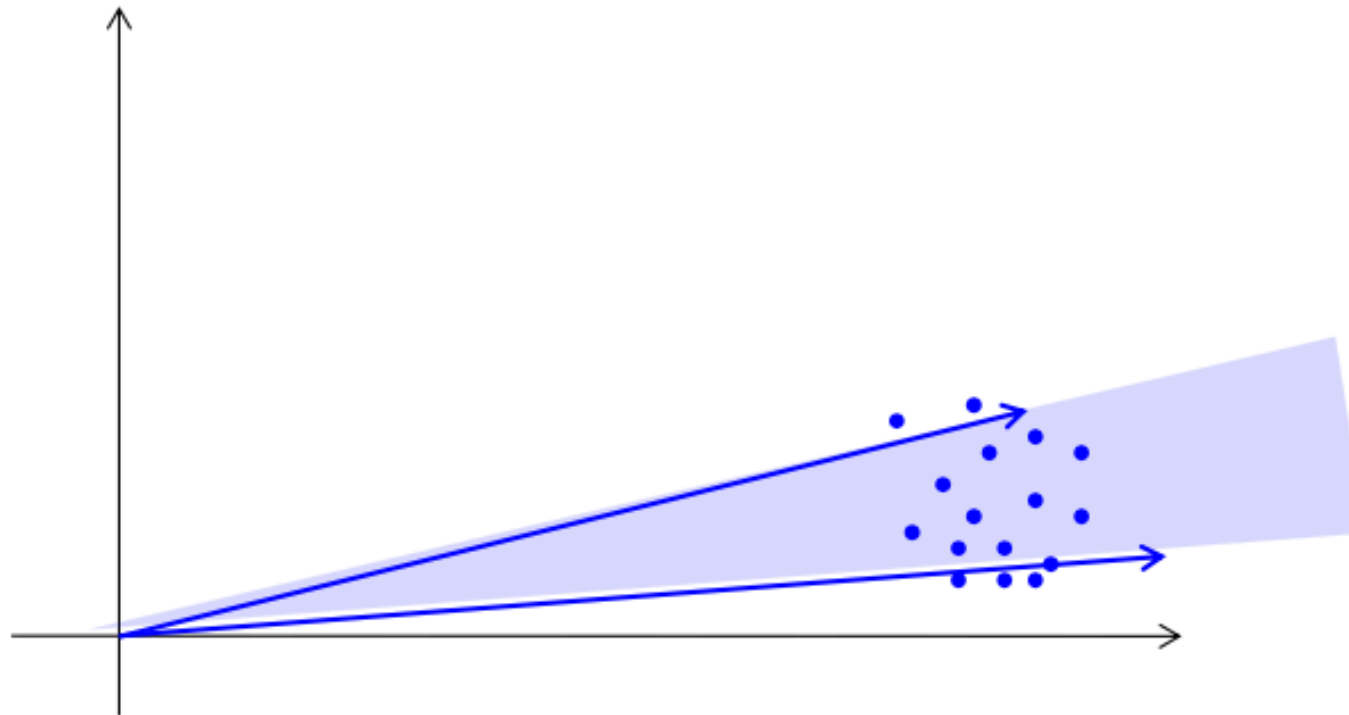


- If we approximate the red dot as a non-negative combination of the bases, the approximation will lie in the shaded region
 - On or close to the boundary
 - The approximation has error

The NMF representation

- The representation characterizes all data as lying within a compact convex region
 - “Compact” → enclosing only a small fraction of the entire space
 - The more compact the enclosed region, the more it localizes the data within it
 - Represents the boundaries of the distribution of the data better
 - Conventional statistical models represent the mode of the distribution
- The *bases* must be chosen to
 - Enclose the data as compactly as possible
 - And also enclose as much of the data as possible
 - Data that are not enclosed are not represented correctly

NMF: Learning Bases

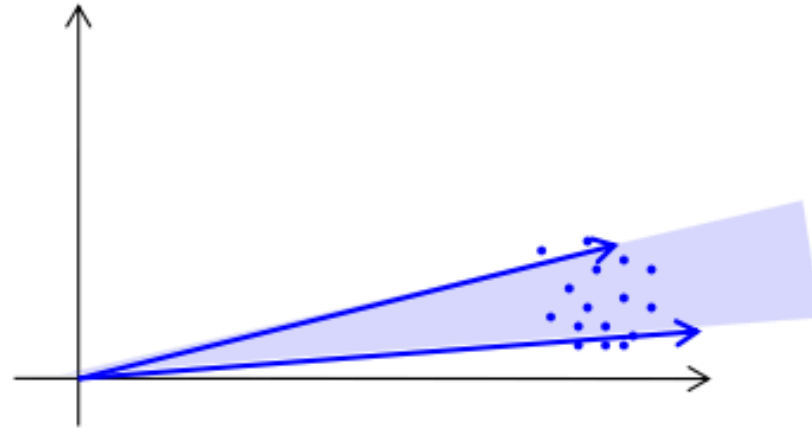


- Given a collection of data vectors (blue dots)
- Goal: find a set of bases (blue arrows) such that they enclose the data.
- Ideally, they must simultaneously enclose the smallest volume
 - *This “enclosure” constraint is usually not explicitly imposed in the standard NMF formulation*

NMF: Learning Bases

- Express every training vector as non-negative combination of bases
 - $V = \sum_i w_i B_i$
- In linear algebraic notation, represent:
 - Set of all training vectors as a data matrix **V**
 - A $D \times N$ matrix, D = dimensionality of vectors, N = No. of vectors
 - All basis vectors as a matrix **B**
 - A $D \times K$ matrix, K is the number of bases
 - The K weights for any vector V as a $K \times 1$ column vector W
 - The weight vectors for all N training data vectors as a matrix **W**
 - $K \times N$ matrix
- Ideally **$V = BW$**

NMF: Learning Bases



- $\mathbf{V} = \mathbf{BW}$ will only hold true if all training vectors in $\mathbf{V}$ lie inside the region enclosed by the bases
- Learning bases is an iterative algorithm
- Intermediate estimates of $\mathbf{B}$ do not satisfy $\mathbf{V} = \mathbf{BW}$
- Algorithm updates $\mathbf{B}$ until $\mathbf{V} = \mathbf{BW}$ is satisfied as closely as possible

NMF: Minimizing Divergence

- Define a *Divergence* between data **V** and approximation **BW**
 - Divergence(**V**, **BW**) is the total error in approximating all vectors in **V** as **BW**
 - Must estimate **B** and **W** so that this error is minimized
- Divergence(**V**, **BW**) can be defined in different ways
 - L2: Divergence = $\sum_i \sum_j (V_{ij} - (BW)_{ij})^2$
 - Minimizing the L2 divergence gives us an algorithm to learn **B** and **W**
 - KL: Divergence(**V**, **BW**) = $\sum_i \sum_j V_{ij} \log(V_{ij} / (BW)_{ij}) + \sum_i \sum_j V_{ij} - \sum_i \sum_j (BW)_{ij}$
 - This is a *generalized* KL divergence that is minimum when **V** = **BW**
 - Minimizing the KL divergence gives us another algorithm to learn **B** and **W**
- Other divergence forms can also be used

NMF: Minimizing L_2 Divergence

- Divergence(**V**, **BW**) is defined as
 - $E = ||\mathbf{V} - \mathbf{BW}||_F^2$
 - $E = \sum_i \sum_j (V_{ij} - (\mathbf{BW})_{ij})^2$
- Iterative solution: Minimize E such that **B** and **W** are strictly non-negative

NMF: Minimizing L2 Divergence

- Learning both **B** and **W** with non-negativity
- Divergence(**V**, **BW**) is defined as
 - $E = ||\mathbf{V} - \mathbf{BW}||_F^2$
 - $$\mathbf{V} \approx \mathbf{BW}$$
- Iterative solution:
 - $\mathbf{B} = [\mathbf{V} \text{Pinv}(\mathbf{W})]_+$
 - $\mathbf{W} = [\text{Pinv}(\mathbf{B}) \mathbf{V}]_+$
 - Subscript + indicates thresholding –ve values to 0

NMF: Minimizing Divergence

- Define a *Divergence* between data **V** and approximation **BW**
 - Divergence(**V**, **BW**) is the total error in approximating all vectors in **V** as **BW**
 - Must estimate **B** and **W** so that this error is minimized
- Divergence(**V**, **BW**) can be defined in different ways
 - L2: Divergence = $\sum_i \sum_j (V_{ij} - (BW)_{ij})^2$
 - Minimizing the L2 divergence gives us an algorithm to learn **B** and **W**
 - KL: Divergence(**V**, **BW**) = $\sum_i \sum_j V_{ij} \log(V_{ij} / (BW)_{ij}) + \sum_i \sum_j V_{ij} - \sum_i \sum_j (BW)_{ij}$
 - This is a *generalized* KL divergence that is minimum when **V** = **BW**
 - Minimizing the KL divergence gives us another algorithm to learn **B** and **W**

- For speech signals and sound processing in general, NMF-based representations work best when we minimize the KL divergence

NMF: Minimizing KL Divergence

- Divergence(**V**, **BW**) defined as
 - $E = \sum_i \sum_j V_{ij} \log(V_{ij} / (BW)_{ij}) + \sum_i \sum_j V_{ij} - \sum_i \sum_j (BW)_{ij}$
- Iterative update rules
- Number of iterative update rules have been proposed
- The most popular one is the multiplicative update rule..

NMF Estimation: Learning bases

- The algorithm to estimate **B** and **W** to minimize the KL divergence between **V** and **BW**:
- Initialize **B** and **W** (randomly)
- Iteratively update **B** and **W** using the following formulae

$$B = B \otimes \frac{\left(\frac{V}{BW}\right)W^T}{1W^T}$$

$$W = W \otimes \frac{B^T\left(\frac{V}{BW}\right)}{B^T1}$$

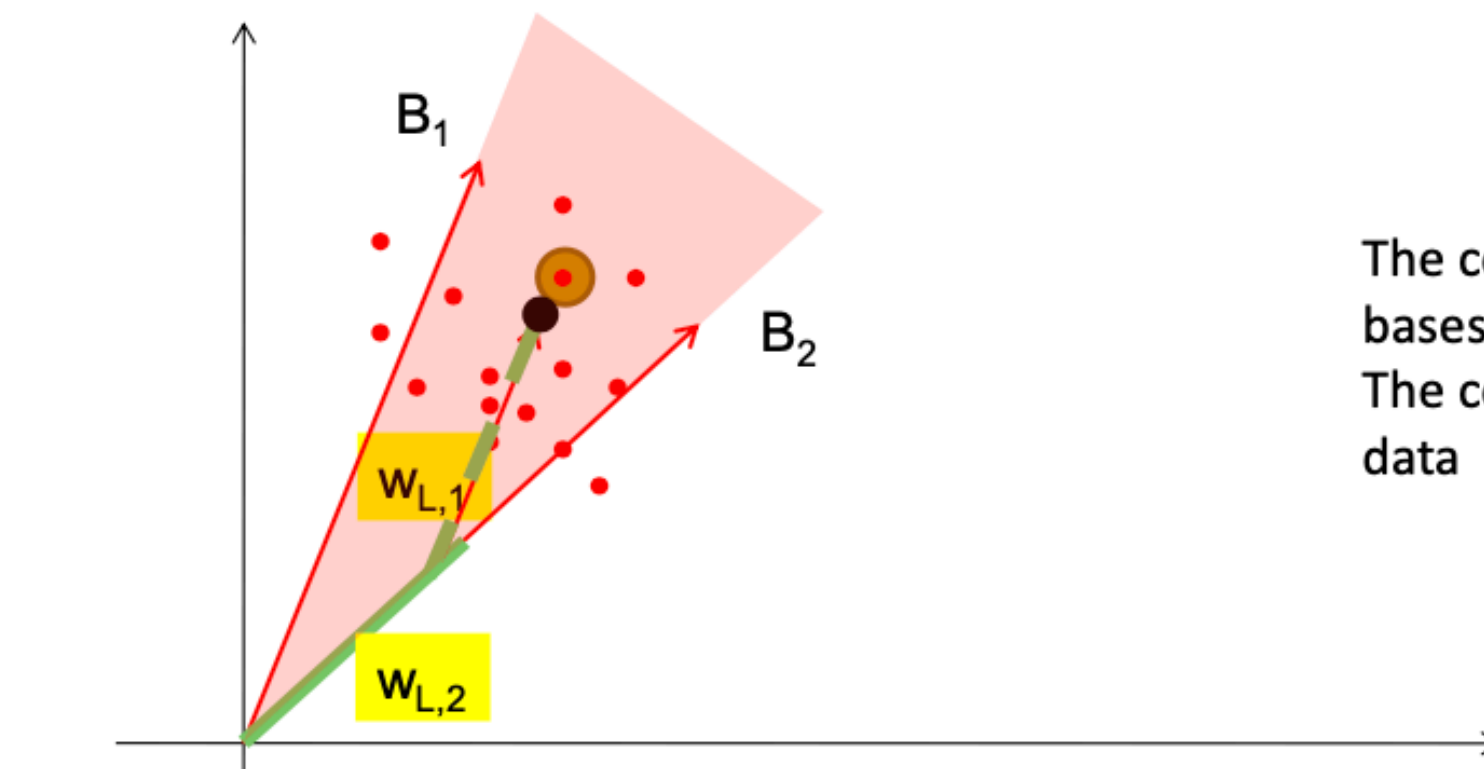
- Iterations continue until divergence converges
 - In practice, continue for a fixed no. of iterations

Reiterating

$$V_{D \times N} \approx B_{D \times K} W_{K \times N}$$

$$V_L \approx \sum_k w_{L,k} B_k$$

- NMF learns the *optimal set of basis vectors* B_k to approximate the data in terms of the bases
- It also learns how to compose the data in terms of these bases
 - Compositions can be inexact



The columns of **B** are the
bases
The columns of **V** are the
data

Gaussian NMF

- Gaussian NMF is the most basic form of non-negative factorizations:

$$\begin{array}{ll}\text{minimize} & \|\mathbf{D} - \mathbf{LR}\|_F^2 \\ \text{s. t.} & \mathbf{L} \in \mathbb{R}_+^{m \times r} \\ & \mathbf{R} \in \mathbb{R}_+^{r \times n}\end{array}$$

- Truncated SVD minimizes the same objective (but without non-negativity constraints)
- Many other variants exist
 - ▶ Different objective functions (e.g., KL-divergence)
 - ▶ Additional regularizations (e.g., L_1 -regularization)
 - ▶ Different constraints (e.g., orthogonality of $\mathbf{R}$)
 - ▶ Different compositions (e.g., 3 matrices)
 - ▶ multi-layer NMF, semi-NMF, sparse NMF, tri-NMF, symmetric NMF, orthogonal NMF, non-smooth NMF (nsNMF), overlapping NMF, convolutive NMF (CNMF), k-Means, ...

General framework

- Gradient descent generally slow
- Stochastic gradient descent inappropriate
- Key approach: alternating minimization
 - 1: Pick starting point $\mathbf{L}_0$ and $\mathbf{R}_0$
 - 2: **while** not converged **do**
 - 3: Keep $\mathbf{R}$ fixed, optimize $\mathbf{L}$
 - 4: Keep $\mathbf{L}$ fixed, optimize $\mathbf{R}$
 - 5: **end while**
- Update steps 3 and 4 easier than full problem
- Also called *alternating projections* or *(block) coordinate descent*
- Starting point
 - ▶ Random
 - ▶ Multi-start initialization: try multiple random starting points, run a few epochs, continue with best
 - ▶ Based on SVD
 - ▶ ...

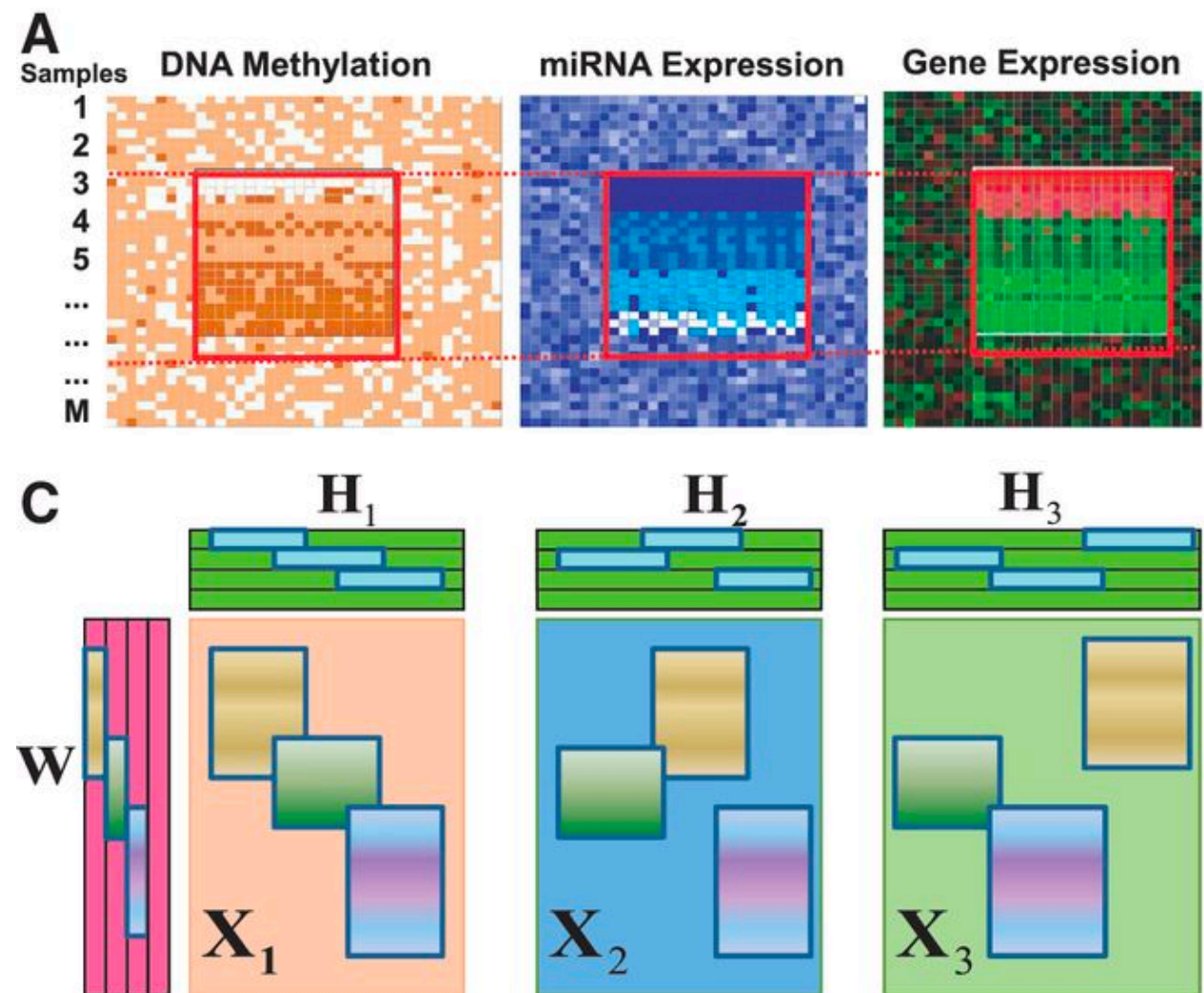
Outline

- Matrix factorization
 - SVD & NMF
- **NMF in bioinformatics**
- Imputation
 - Proteomics
 - Single cell RNA-seq
- Dimension reduction
 - Overview
 - Isomap, t-SNE and UMAP
 - 3D reconstruction from single cell RNA-seq

NMF in bioinformatics - example 1

Discovery of multi-dimensional modules by integrative analysis of cancer genomic data

for the discovery of **combinatorial patterns in 'multi-dimensional genomic data'**: multi-platform genomic profiling (e.g. DNA methylation (DM) and gene expression (GE)) of biological samples.



An example of md-modules. In the three data matrices, rows correspond to the samples and columns correspond to different measurements. An md-module consists of r rows and n_i ($i = 1, 2, 3$) columns for GE (Gene Expression), ME (miRNA Expression) and DM (DNA Methylation) data, respectively. These subsets of DMs, MEs and GEs exhibit correlated profiles across a subset of samples.

The joint NMF framework for integrative analysis

Let X_1 , X_2 and X_3 be $M \times N_1$, $M \times N_2$ and $M \times N_3$ matrices representing three types of genomic profiling of the same samples, e.g. the methylation profiles of N_1 DNA markers and the expressions of N_2 genes and N_3 miRNAs of M samples. To extract md-modules across the three data matrices, the following joint factorization framework was used to decompose the three data matrices into a common **basis matrix** W and different **coefficient matrices** H_I ($I = 1, 2, 3$):

$$X_I \approx WH_I,$$

with the non-negativity constraints:

$$W \geq 0, \quad H_I \geq 0, \quad I = 1, 2, 3,$$

where W is an $M \times K$ matrix, and each column of W represents a basis vector of the reduced system. H_I is a matrix of size $K \times N_I$, and each row of H_I represents a coefficient vector. Then, the objective function of joint NMF is formulated as

$$\min \sum_{I=1}^3 \|X_I - WH_I\|_F^2.$$

Several algorithms have been developed to optimize the NMF problem (19). Lee and Seung (18) devised a multiplicative algorithm that is simple to implement and performs well. Like the standard NMF, we used the ‘**multiplicative update**’ equations to **minimize the Euclidean error function**. Specifically, given a desired rank K , the algorithm iteratively computes the approximations of X_1 , X_2 and X_3 in the same manner. The method starts by randomly initializing matrices W and H_1 , H_2 and H_3 , which are iteratively updated to minimize the Euclidean distance function. Specifically, W , H_1 , H_2 and H_3 are updated at each step by using the generalized multiplicative update rules as follows:

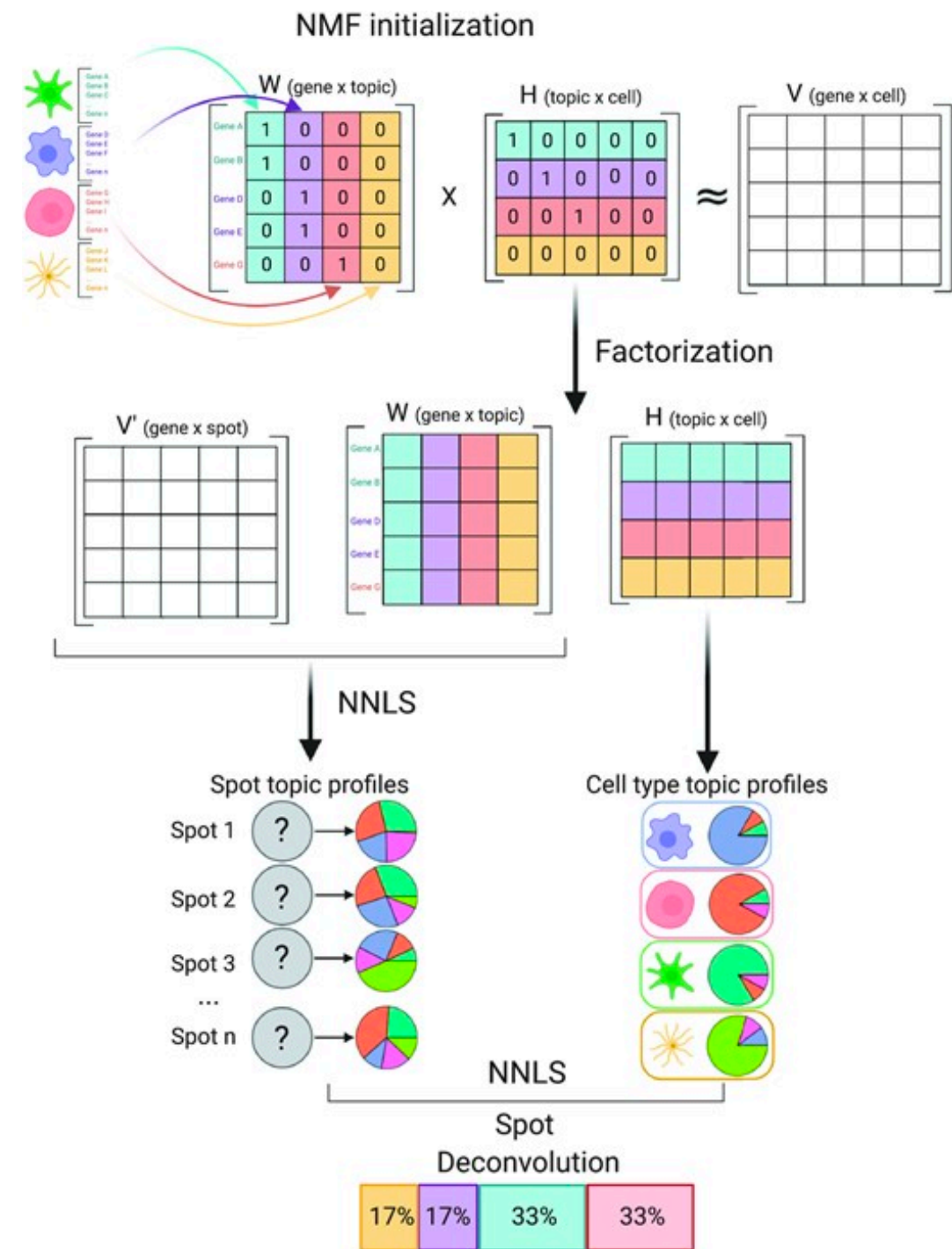
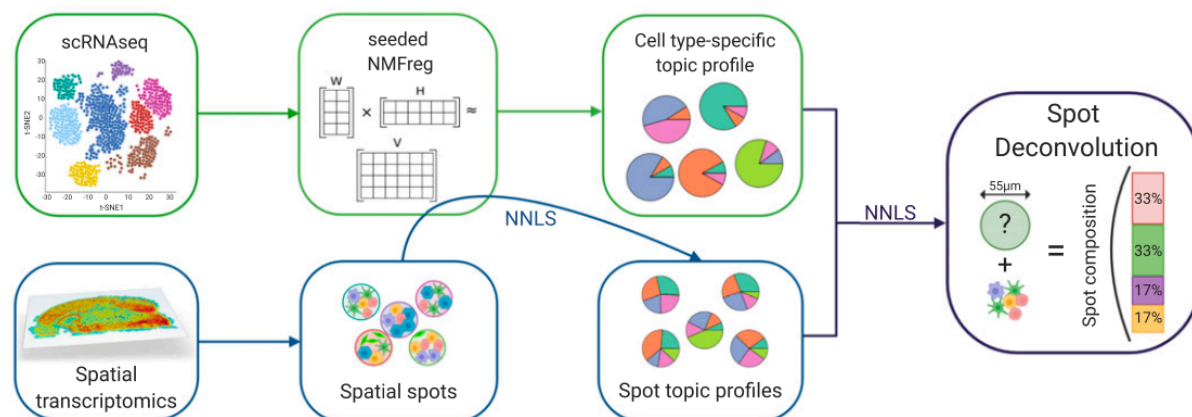
$$W_{ia} = W_{ia} \frac{(X_1 H_1^T + X_2 H_2^T + X_3 H_3^T)_{ia}}{(W(H_1 H_1^T + H_2 H_2^T + H_3 H_3^T))_{ia}},$$

$$(H_I)_{a\mu} = (H_I)_{a\mu} \frac{(W^T X_I)_{a\mu}}{(W^T W H_I)_{a\mu}}, \quad I = 1, 2, 3.$$

NMF in bioinformatics - example 2

SPOTlight: seeded NMF regression to deconvolute spatial transcriptomics spots with single-cell transcriptomes

Spatial Transcriptomics (ST) technologies generate gene expression profiles while retaining the tissue context. However, most methods lack single-cell resolution and encompass multiple cells within their capture sites (spots). SPOTlight uses **seeded NMF regression** to integrate single-cell RNA sequencing and ST datasets. SPOTlight learns topic **signatures** from single-cell data and finds the **optimal weighted** combinations of cell types to **explain a spot's cellular composition**.



<https://academic.oup.com/nar/article/49/9/e50/6129341>

The SPOTlight code and the analysis notebooks to re-produce the aforementioned analysis are hosted at <https://github.com/MarcElosua/SPOTlight> and <https://github.com/MarcElosua/SPOTlight> deconvolution analysis.

At the core of our tool, we use non-negative matrix factorization (NMF) along with non-negative least squares (NNLS). NMF is used to factorize a matrix into two or more lower dimensionality matrices without negative elements. We first have an initial matrix V , which is factored into W and H . Unit variance normalization by gene is performed in V and V' in order to standardize discretized gene expression levels, ‘counts-umi’ (10,12). Factorization is then carried out using the non-smooth NMF method (13), implemented in the R package *NMF* (14). This method is intended to return sparser results during the factorization in W and H , thus promoting cell-type-specific topic profile and reducing overfitting during training. Before running factorization, we initialize each topic, column, of W with the unique marker genes for each cell type with weights $1 - P$ value. The marker genes are obtained from Seurat’s function *FindAllMarkers*. In turn, each topic of H in SPOTlight is initialized with the corresponding belongingness of each cell for each topic, 1 or 0. This way, we seed the model with prior information, thus guiding it towards a biologically relevant result. This initialization also aims at reducing variability and improving the consistency between runs.

$$V \sim W * H$$

Second, NNLS regression is used to map each capture location’s transcriptome in V' to H' using W as the basis. We obtain a topic profile distribution over each capture location which we can use to determine its composition.

$$V' \sim W * H'$$

Third, we obtain Q , cell-type specific topic profiles, from H . We select all cells from the same cell type and compute the median of each topic for a consensus cell-type-specific topic signature. We then use NNLS to find the weights of each cell type that best fit H' minimizing the residuals.

$$H' \sim Q * P$$

- N – Set of all cells from scRNAseq.
- M – Set of all capture locations from spatial data.
- G – Set of selected genes from scRNAseq, cell type marker genes + 3000 highly variable genes.

We use a minimum weight contribution to determine which cell types belong within a capture location. 0.09% is set by default, related to the expected number of cells at the capture locations (1–10 cells). In a scenario with 10 cells, we would detect all and also account for partially contributing cells.

By using NNLS, we are able to return a measure of error along with the predicted cell proportions. To do so, we calculate the total sum of squares (TSS) and the residual sum of squares (RSS) for each row. By dividing the RSS by the TSS we obtain the percentage of unexplained residuals for each spot. This measure can be used to assess the quality of a predicted composition.

$$TSS = \sum_{i=1}^C (Y_i - 0)^2$$

$$RSS = \sum_{i=1}^C (Y_i - \hat{Y}_i)^2$$

$$\text{Unexplained residuals (\%)} = RSS/TSS$$

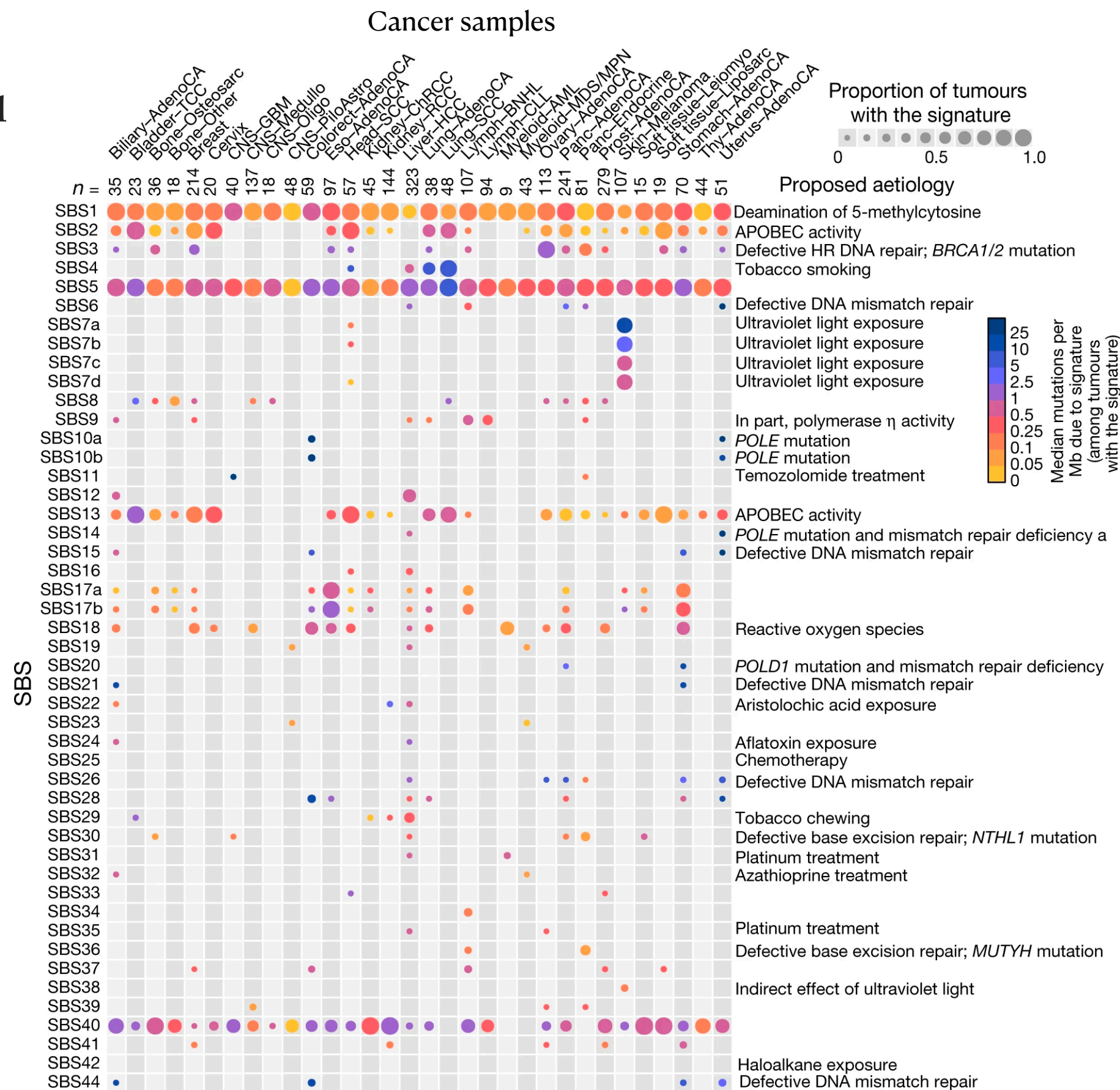
- V – matrix of dimensions $G_i \times N$ containing data from scRNAseq
- W – matrix of dimension $G_i \times K$ containing the gene distribution for each topic, basis between V and H .
- H – matrix of dimensions $K \times N$ containing the topic distribution for each cell.
- V' – matrix of dimensions $G_i \times M$ containing spatial data.
- H' – matrix of dimensions $K \times M$ containing the topic distributions for each capture location.
- Q – matrix of dimension $K \times C$ containing the topic distributions for each cell type.
- P – matrix of dimension $C \times M$ containing the cell type weights for each capture location.
- G' – Set of all genes from spatial data.
- $G_i = G \cap G'$, intersection between G and G' .
- C – Number of cell types in the scRNAseq dataset
- K – Number of topics to use to reduce the dimensionalities, equal to C .

NMF in bioinformatics - example 3

The repertoire of mutational signatures in human cancer

Fig. The number of mutations contributed by each mutational signature to the PCAWG tumours.

Mutation signatures



Analysis of mutational signatures with SigProfiler

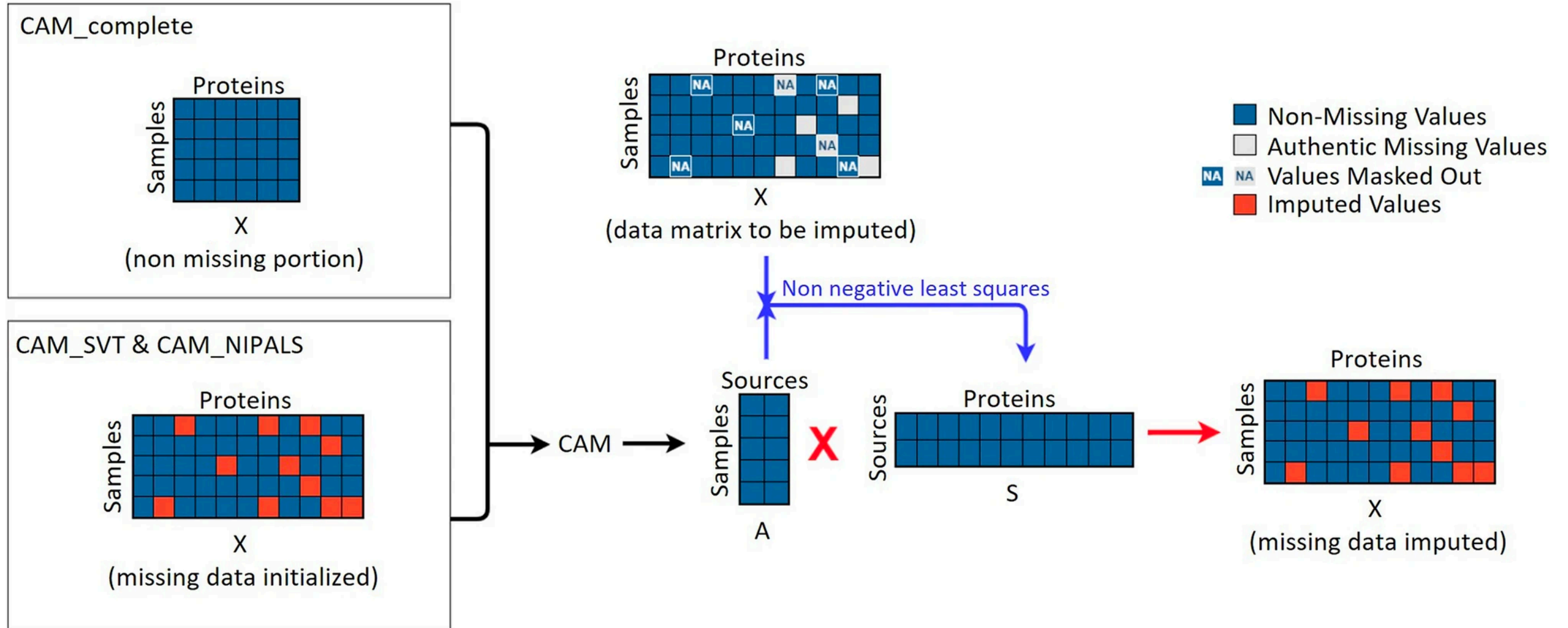
SigProfiler incorporates two distinct steps for identification of mutational signatures, based on the previously described methodology^{6,11,17} (Extended Data Fig. [8](#)). The first step (SigProfilerExtraction) encompasses a hierarchical de novo extraction of mutational signatures based on somatic mutations and their immediate sequence context, and the second step (SigProfilerAttribution) focuses on accurately estimating the number of somatic mutations associated with each extracted mutational signature in each sample. SigProfilerExtraction is an extension of a previous framework for the analysis of mutational signatures^{11,17}.

In brief, for a given set of mutational catalogues, the algorithm deciphers a minimal set of mutational signatures that optimally explains the proportion of each mutation type and estimates the contribution of each signature to each sample. More specifically, for each NMF iteration, SigProfilerExtraction minimizes a generalized Kullback–Leibler divergence constrained for nonnegativity ([Supplementary Note 2](#)). The algorithm uses multiple NMF iterations (in most cases 1,024) to identify the matrix of mutational signatures and the matrix of the activities of these signatures, as previously described¹⁷. The unknown number of signatures is determined by human assessment of the stability and accuracy of solutions for a range of values, as previously described¹⁷. The framework is **applied hierarchically** to increase its ability to find mutational signatures that generate few mutations or are present in few samples.

After signatures are discovered by SigProfilerExtraction, SigProfilerAttribution **estimates their contributions** to individual samples. For each examined sample, the estimation algorithm involves finding the minimum of the **Frobenius norm** of a constrained function using a nonlinear convex optimization programming solver using the interior-point algorithm⁶³. See [Supplementary Note 2](#) and Extended Data Fig. [8b](#) for further details.

Outline

- Matrix factorization
 - SVD & NMF
- NMF in bioinformatics
- Imputation
 - **Proteomics**
 - Single cell RNA-seq
- Dimension reduction
 - Overview
 - Isomap, t-SNE and UMAP
 - 3D reconstruction from single cell RNA-seq



Introduction to CAM method

CAM is a latent variable modeling and deconvolution technique previously used for identifying biologically-interpretable cell subtypes or biological archetypes $\mathcal{S}_{l \times n}$ and their composition $\mathcal{A}_{m \times l}$ in complex tissue ecosystems^{6,14,15,21}. We adopted the CAM framework into Eq. 1 and demonstrate that hybrid CAM_SVT and CAM_NIPALS can effectively manage missing values and that this combination leads to a novel and biologically-plausible imputation strategy. The workflow of CAM based method with three variants is given in Fig. 5. Below we present a brief introduction to the CAM principle. Detailed mathematical descriptions and algorithms can be found in our previous publications^{14,15}.

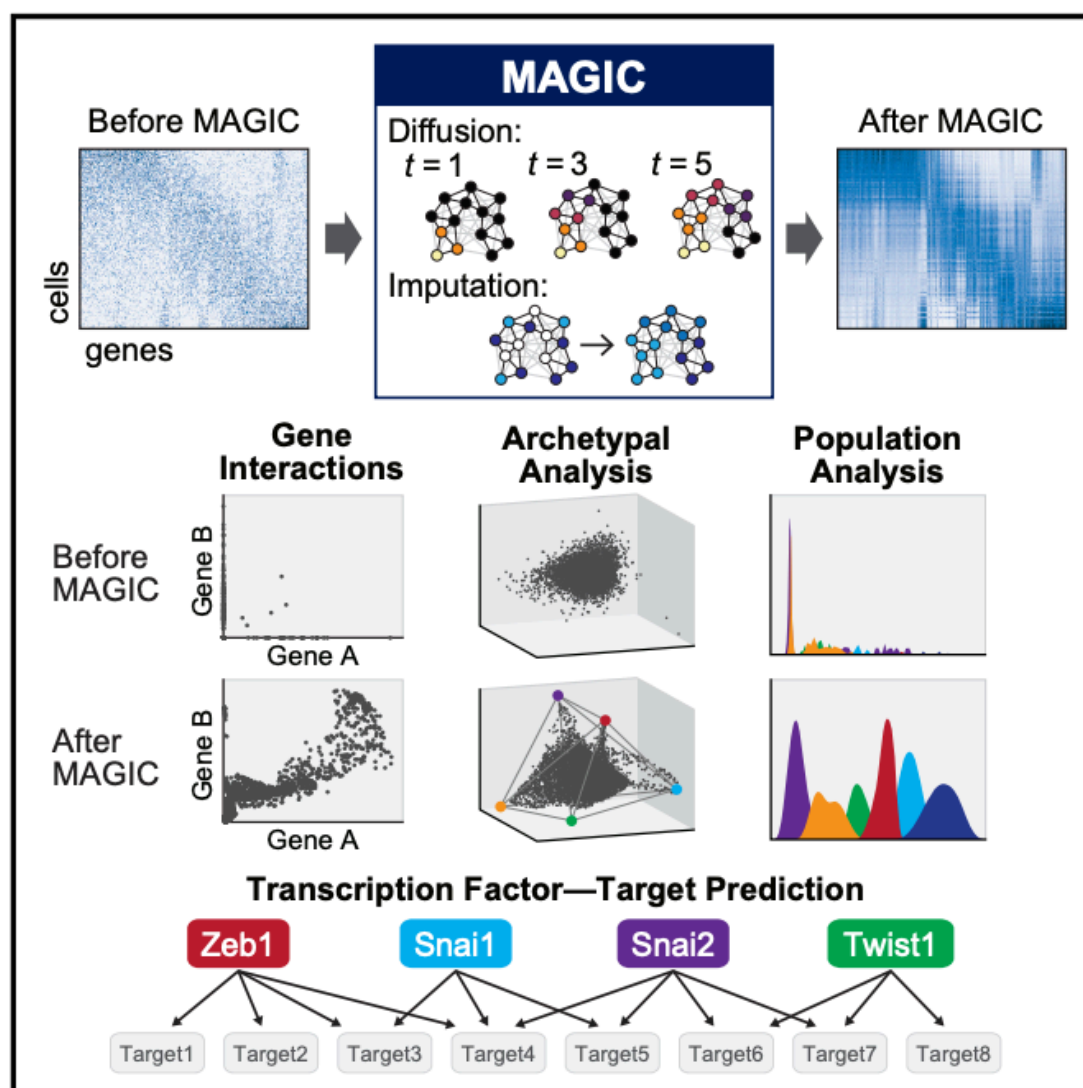
Imputation for scRNA-seq example1 - MAGIC

Resource

Cell

Recovering Gene Interactions from Single-Cell Data Using Data Diffusion

Graphical Abstract



Authors

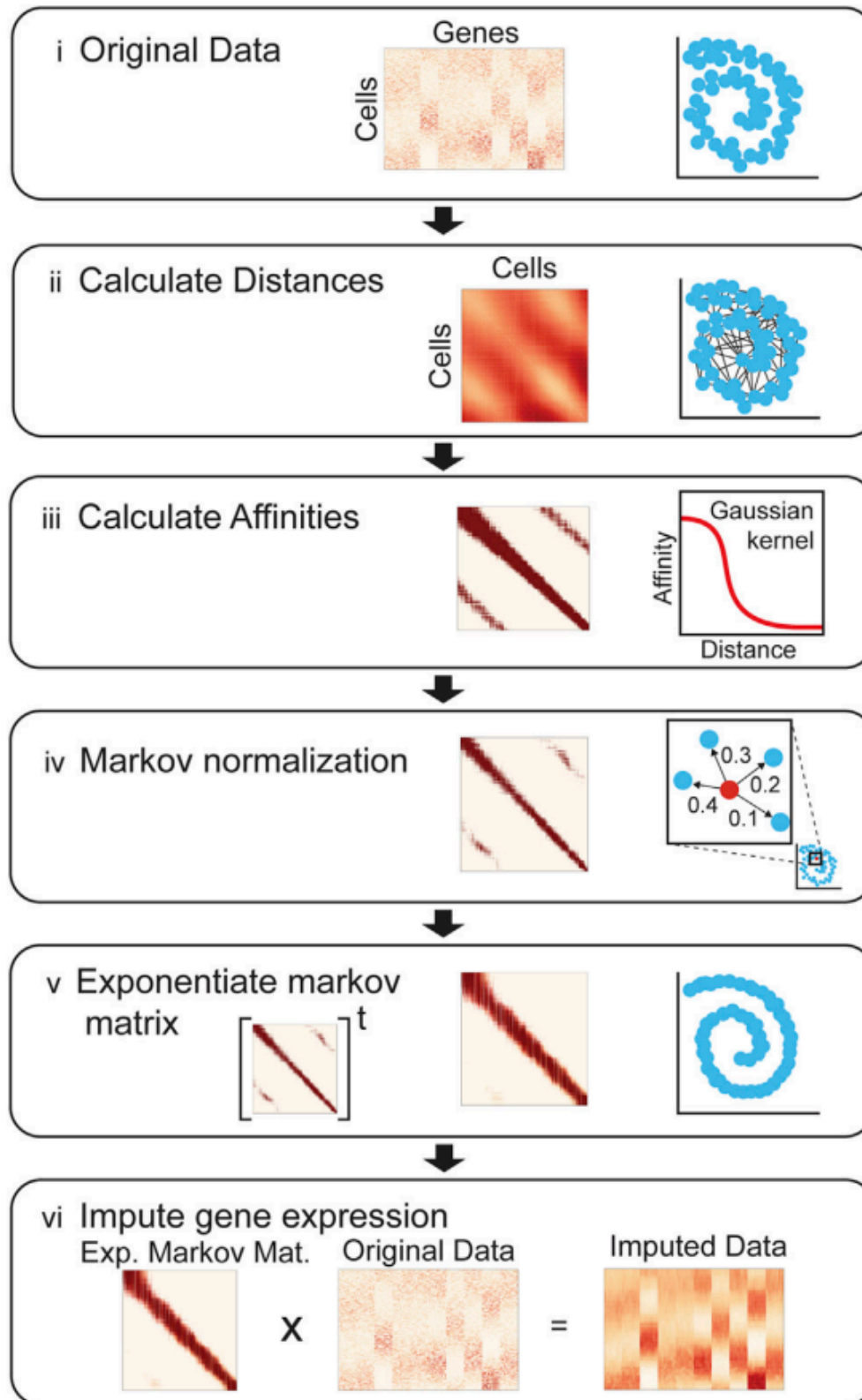
David van Dijk, Roshan Sharma,
Juozas Nainys, ..., Guy Wolf,
Smita Krishnaswamy, Dana Pe'er

Correspondence

smita.krishnaswamy@yale.edu (S.K.),
peerd@mskcc.org (D.P.)

In Brief

A new algorithm overcomes limitations of
data loss in single-cell sequencing
experiments.



$MAGIC(D, t)$

$D = preprocess(D)$

$Dist = compute_distance_matrix(D)$

$A = compute_affinity_matrix(Dist)$

$M = compute_markov_affinity_matrix(A)$

$D_{imputed} = M^t * D$

$D_{rescaled} = Rescale(D_{imputed})$

$D_{imputed} = D_{rescaled}$

END

Imputation for scRNA-seq example2 - scImpute



DOI: 10.1038/s41467-018-03405-7

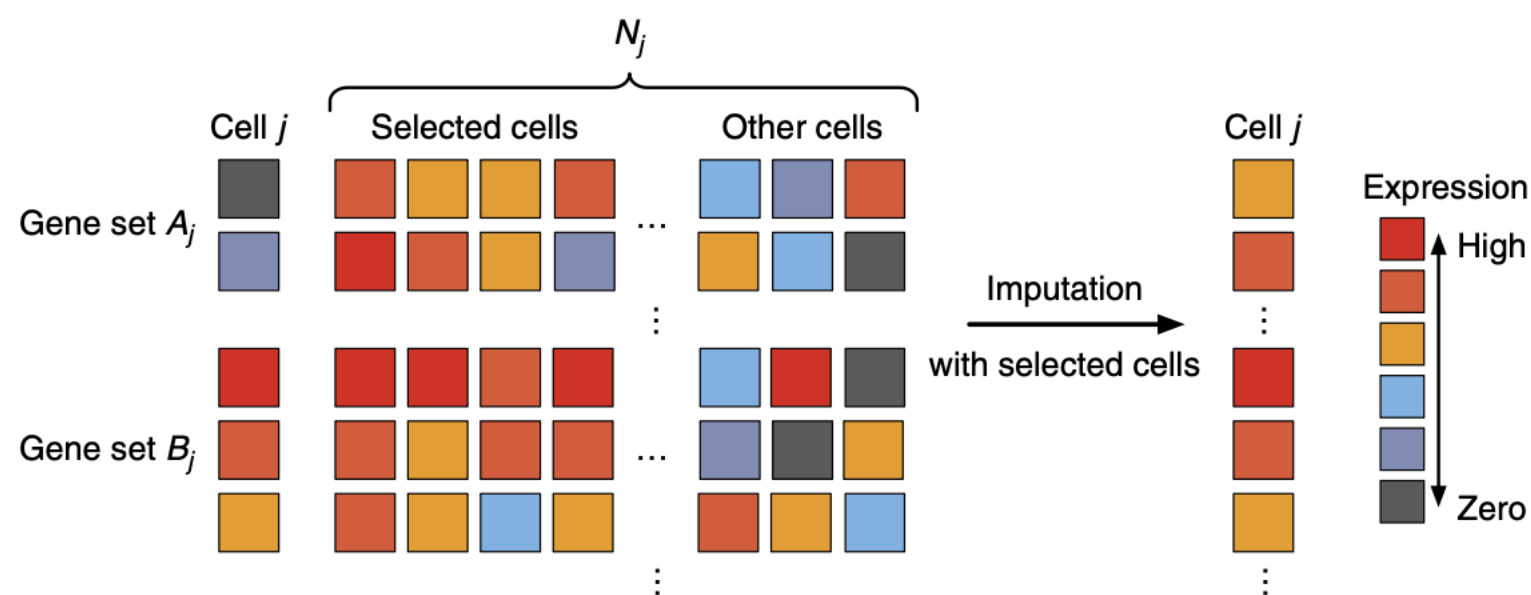
ARTICLE

DOI: 10.1038/s41467-018-03405-7

OPEN

An accurate and robust imputation method
scImpute for single-cell RNA-seq data

Wei Vivian Li¹ & Jingyi Jessica Li^{1,2}



$$\hat{\beta}^{(j)} = \operatorname{argmin}_{\beta^{(j)}} \left\| \mathbf{X}_{B_j, j} - \mathbf{X}_{B_j, N_j} \beta^{(j)} \right\|_2^2, \text{ subject to } \beta^{(j)} \geq \mathbf{0}. \quad (2)$$

$$\hat{X}_{ij} = \begin{cases} X_{ij}, & i \in B_j, \\ X_{i, N_j} \hat{\beta}^{(j)}, & i \in A_j. \end{cases} \quad (3)$$

Fig. 1 A toy example illustrating the workflow in the imputation step of scImpute method. scImpute first learns each gene's dropout probability in each cell by fitting a mixture model. Next, scImpute imputes the (highly probable) dropout values in cell j (gene set A_j) by borrowing information of the same gene in other similar cells, which are selected based on gene set B_j (not severely affected by dropout events). The details are described in Eqs. (2) and (3)

Identification of dropout values. Once we obtain the transformed gene expression matrix X and the candidate neighbors of each cell N_j , the next step is to infer which genes are affected by the dropout events in which cells. Instead of treating all zero values as dropout events, we construct a statistical model to systematically determine whether a zero value comes from a dropout event or not. With the existence of dropout events, most genes have a bimodal expression pattern across similar cells, and that pattern can be described by a mixture model of two components (Supplementary Fig. 18). The first component is a Gamma distribution used to account for the dropouts, while the second component is a Normal distribution to represent the actual gene expression levels. Please note that the transformed gene expression levels are no longer integers, so the widely used negative binomial distribution for read counts is not a proper choice here. For each gene, the proportions and parameters of the two components could be different in various cell types, so we construct separate mixture models for different cell subpopulations.

For each gene i , its expression in cell subpopulation k is modeled as a random variable $X_i^{(k)}$ with density function

$$f_{X_i^{(k)}}(x) = \lambda_i^{(k)} \text{Gamma}(x; \alpha_i^{(k)}, \beta_i^{(k)}) + (1 - \lambda_i^{(k)}) \text{Normal}(x; \mu_i^{(k)}, \sigma_i^{(k)}), \quad (1)$$

where $\lambda_i^{(k)}$ is gene i 's dropout rate in cell subpopulation k , $\alpha_i^{(k)}, \beta_i^{(k)}$ are the shape and rate parameters of Gamma distribution, and $\mu_i^{(k)}, \sigma_i^{(k)}$ are the mean and standard deviation of Normal distribution. The intuition behind this mixture model is that if a gene has high expression and low variation in most cells within a cell subpopulation, a zero count is more likely to be a dropout value; on the other hand, if a gene has constantly low or medium expression with high variation, then a zero count may reflect real biological variability. An advantage of this model is that it does not assume an empirical relationship between dropout rates and mean gene expression levels, as Kharchenko et al.⁷ did, thus allowing more flexibility in the model estimation. The parameters in the mixture model are estimated by the Expectation–Maximization (EM) algorithm, and we denote their estimates as $\hat{\lambda}_i^{(k)}$, $\hat{\alpha}_i^{(k)}$, $\hat{\beta}_i^{(k)}$, $\hat{\mu}_i^{(k)}$, and $\hat{\sigma}_i^{(k)}$. It follows that the dropout probability of gene i in cell j , which belongs to subpopulation k , can be estimated as

$$d_{ij} = \frac{\hat{\lambda}_i^{(k)} \text{Gamma}(X_{ij}; \hat{\alpha}_i^{(k)}, \hat{\beta}_i^{(k)})}{\hat{\lambda}_i^{(k)} \text{Gamma}(X_{ij}; \hat{\alpha}_i^{(k)}, \hat{\beta}_i^{(k)}) + (1 - \hat{\lambda}_i^{(k)}) \text{Normal}(X_{ij}; \hat{\mu}_i^{(k)}, \hat{\sigma}_i^{(k)})}.$$

Therefore, each gene i has an overall dropout rate $\hat{\lambda}_i^{(k)}$ in cell subpopulation k , which does not depend on individual cells within the subpopulation. Gene i also has dropout probabilities d_{ij} ($j = 1, 2, \dots, J$), which may vary among different cells. During the preparation of this manuscript, it came to our attention that Ghazanfar et al.³⁸ also used the Gamma-Normal mixture model to analyze scRNA-seq data but only applied it to categorize non-zero expression values into low-expression values and high-expression values.

Imputation of dropout values. Now, we impute the gene expressions cell by cell. For each cell j , we select a gene set A_j in need of imputation based on the genes' dropout probabilities in cell j : $A_j = \{i : d_{ij} \geq t\}$, where t is a threshold on dropout probabilities. We also have a gene set $B_j = \{i : d_{ij} < t\}$ that have accurate gene expression with high confidence and do not need imputation. We learn cells' similarities through the gene set B_j . Then we impute the expression of genes in the set A_j by borrowing information from the same gene's expression in other similar cells learned from B_j . Supplementary Figs. 19 and 20c give some real data distributions of genes' zero count proportions across cells and genes' dropout probabilities, showing that it is reasonable to divide genes into two sets. To learn the cells similar to cell j from B_j , we use the non-negative least squares (NNLS) regression:

$$\hat{\beta}^{(j)} = \text{argmin}_{\beta^{(j)}} \left\| \mathbf{X}_{B_j, j} - \mathbf{X}_{B_j, N_j} \beta^{(j)} \right\|_2^2, \text{ subject to } \beta^{(j)} \geq \mathbf{0}. \quad (2)$$

Recall that N_j represents the indices of cells that are candidate neighbors of cell j . The response $\mathbf{X}_{B_j, j}$ is a vector representing the B_j rows in the j -th column of X , the design matrix $\mathbf{X}_{B_j, N_j}$ is a sub-matrix of X with dimensions $|B_j| \times |N_j|$, and the coefficients $\beta^{(j)}$ is a vector of length $|N_j|$. Note that NNLS itself has the property of leading to a sparse estimate $\hat{\beta}^{(j)}$, whose components may have exact zeros³⁹, so NNLS can be used to select similar cells of cell j from its neighbors N_j . Finally, the estimated coefficients $\hat{\beta}^{(j)}$ from the set B_j are used to impute the expression of genes in the set A_j in cell j :

$$\hat{X}_{ij} = \begin{cases} X_{ij}, & i \in B_j, \\ X_{i, N_j} \hat{\beta}^{(j)}, & i \in A_j. \end{cases} \quad (3)$$

Imputation for scRNA-seq example 3- ALRA



ARTICLE



<https://doi.org/10.1038/s41467-021-27729-z>

OPEN

Zero-preserving imputation of single-cell RNA-seq data

George C. Linderman^{1,8}, Jun Zhao^{2,8}, Manolis Roulis³, Piotr Bielecki^{3,7}, Richard A. Flavell^{3,4}, Boaz Nadler⁵ & Yuval Kluger^{1,2,6}✉

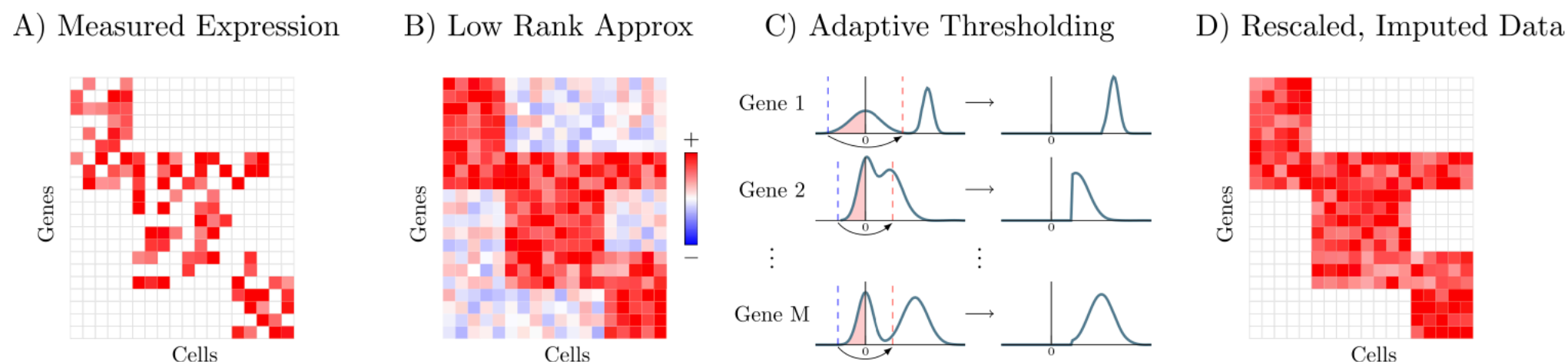


Fig. 1 Overview of the ALRA imputation scheme. **A** Measured expression matrix contains technical zeros (in each block) and biological zeros (outside each block). **B** Low-rank approximation via SVD results in a matrix where every zero is completed with a non-zero value (i.e., biological zeros are not preserved). **C** The elements corresponding to biological zeros for each gene are symmetrically distributed around 0, so the magnitude of the most negative elements in each row is also approximately the magnitude of the most positive values assigned to biological zeros. Therefore, thresholding the values in each row, restores almost all of the biological zeros in the imputed matrix (**D**).

ALRA procedure. Given a measured expression matrix, as outlined in Algorithm 1, ALRA consists of the following steps. First, as commonly done with scRNA-seq data, the first step is standard pre-processing of the expression matrix by library normalization and log-transformation. Specifically, we normalize each column to $\alpha = 10,000$ transcripts per cell, add 1 to every entry (i.e., a “pseudocount”), and take the logarithm of each entry to obtain the normalized expression matrix. It has been noted that different values of α can affect downstream results³⁰, but we show that ALRA’s preservation of biological zeros is not dependent on this choice (Supplementary Tables 6 to 9, Supplementary Fig. 19). As discussed in the Appendix, a variance stabilizing transform is necessary for good downstream results, and we chose the log-transform due to its widespread use. Other kinds of normalization can also likely be used.

After normalization, the next three steps are as follows: (i) Estimate the rank k (ii) compute the rank- k SVD using randomized SVD and (iii) threshold each gene of the rank- k SVD by the absolute value of that gene’s $p = 0.001$ quantile. Finally, we rescale the resulting values such that the mean and standard deviation of the non-zero values of each gene in the resulting matrix match those of the normalized matrix (prior to low-rank approximation). A small number of non-zero values in the normalized matrix may be smaller than the threshold and may thus get set to zero by the thresholding operation in step (iii) of ALRA. For improved consistency with the observed matrix, we restore these entries to their original normalized values. We note that this process not only imputes technical zeros, but also denoises the originally non-zero values. An intuitive measure of how much the non-zero values were adjusted is the Pearson’s correlation between these values before and after ALRA (e.g., for the purified PBMCs and Hrvatin et al. datasets, the correlations were 0.56 and 0.23, respectively).

Implementing ALRA using standard SVD procedures in common software may be rather slow and not scale to large matrices. To allow ALRA to scale to matrices with hundreds of thousands to millions of cells, we apply recently developed methods for randomized SVD³¹. Randomized SVD can be used to compute the leading singular values and singular vectors of large matrices with high accuracy yet at a fraction of the computing time and memory requirements. By default, ALRA uses the single-threaded implementation from the R package rsvd³². For Linux and OS X users, we also developed a multithreaded implementation based on Intel

Math Kernel Library (MKL) that can be installed with one command and enabled by the ‘use.mkl’ argument of the alra function. When imputing data using randomized SVD, we set the parameter for the number of additional power iterations to 10, for increased accuracy.

Algorithm 1. Adaptively thresholded Low-Rank Approximation

Input: Raw expression matrix X , number of rows (genes) m , number of columns (cells) n

Output: Imputed expression matrix Y

Step 0. (Pre-processing) Normalization

for $i \leftarrow 1$ to n do

 Normalize each column to 10,000 transcripts: $X_{:,i} \leftarrow 10,000 (X_{:,i}) / \sum_{j=1}^m X_{j,i}$

 Add a pseudo-count and take the logarithm: $X_{:,i} \leftarrow \log(X_{:,i} + 1)$

end

Step 1. Estimate k .

Compute the rank-100 SVD $\sum_{i=1}^{100} u_i v_i^T \sigma_i$ of X .

Define the spacings $s_i = \sigma_i - \sigma_{i+1}$.

Compute the mean μ and standard deviation σ of $s_{79}, \dots, s_{99}$.

Let $k = \max \{ i \mid s_i > \mu + 6\sigma \}$.

Step 2. Imputation by low-rank approximation.

Compute the rank- k randomized SVD $Y = \sum_{i=1}^k u'_i v_i'^T \sigma'_i$ of X .

Step 3. Restoration of biological zeros.

for $i \leftarrow 1$ to m do

 Let c_i denote the $p = 0.001$ quantile of $Y_{i,:}$

 for $j \leftarrow 1$ to n do

 if $Y_{i,j} < c_i$ then

$Y_{i,j} \leftarrow 0$

 end

 end

end

Step 4. Shifting and rescaling.

for $i \leftarrow 1$ to m do

 Let μ_i, σ_i denote the mean and standard deviation of the non-zero entries of $X_{i,:}$

 Let μ'_i, σ'_i denote the mean and standard deviation of the non-zero entries of $Y_{i,:}$

 Shift and rescale the imputed values

$$Y_{i,:} = (Y_{i,:} - \mu'_i) \cdot \frac{\sigma_i}{\sigma'_i} + \mu_i$$

end

Outline

- Matrix factorization
 - SVD & NMF
- NMF in bioinformatics
- Imputation
 - Proteomics
 - Single cell RNA-seq
- **Dimension reduction**
 - Overview
 - Isomap, t-SNE and UMAP
 - 3D reconstruction from single cell RNA-seq

Dimensionality Reduction

□ Linear methods

- **PCA** (Principal Component Analysis)
- **cMDS** (Classical Multidimensional Scaling)

□ Non-linear methods

- **KPCA** (Kernel PCA)
- **mMDS** (Metric MDS)
- **Isomap**
- **LLE** (Locally Linear Embedding)
- **Laplacian Eigenmap**
- **t-SNE** (t-distributed Stochastic Neighbor Embedding)
- **UMAP** (Uniform Manifold Approximation and Projection)

Trends in Dimensionality Reduction

1901

PCA

1958

MDS

1963

SVM

1964

Kernel Perceptron

1969

Sammon's Mapping

1992

Kernel SVM

1997

Metric MDS

1998

Kernel PCA

2000

Isomap, LLE

2001

Laplacian Eigenmap

2008

t-SNE

2018

UMAP

Global property preserving
Local manifold preserving
Local manifold using Gaussian

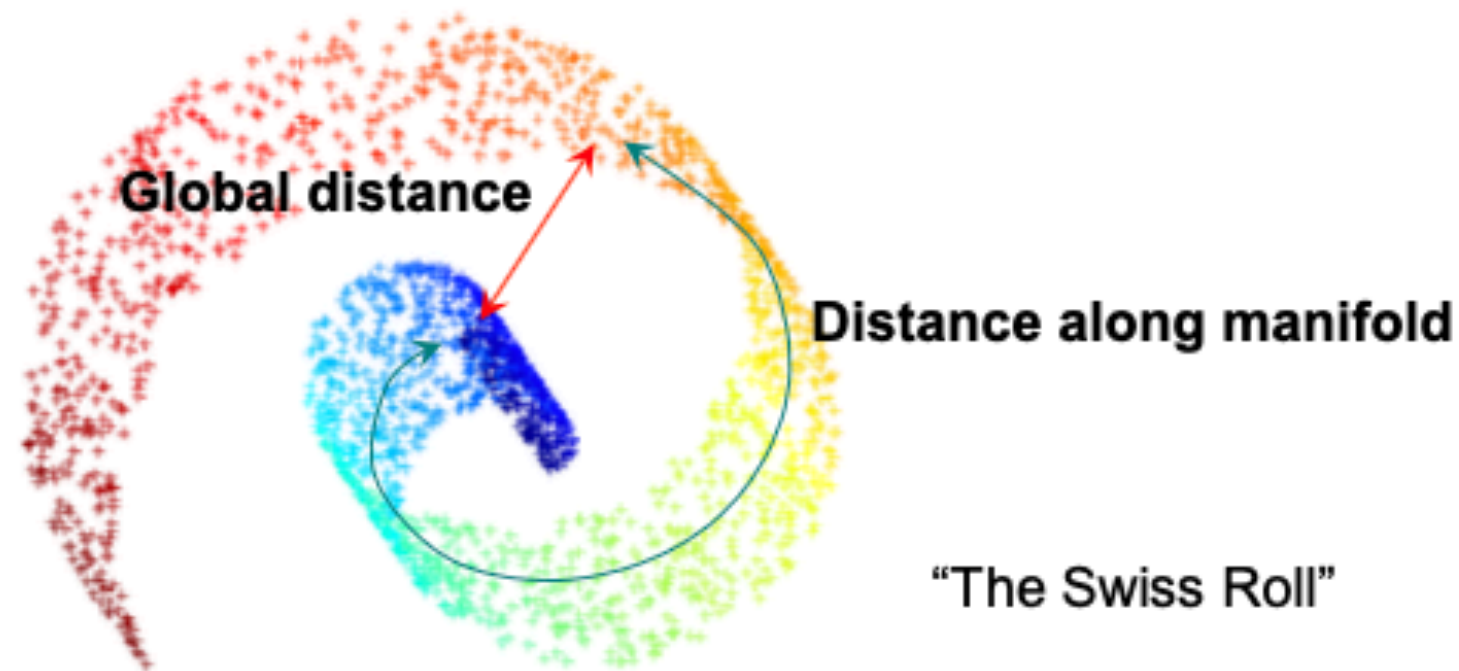
Linear

Non-linear

Non-global

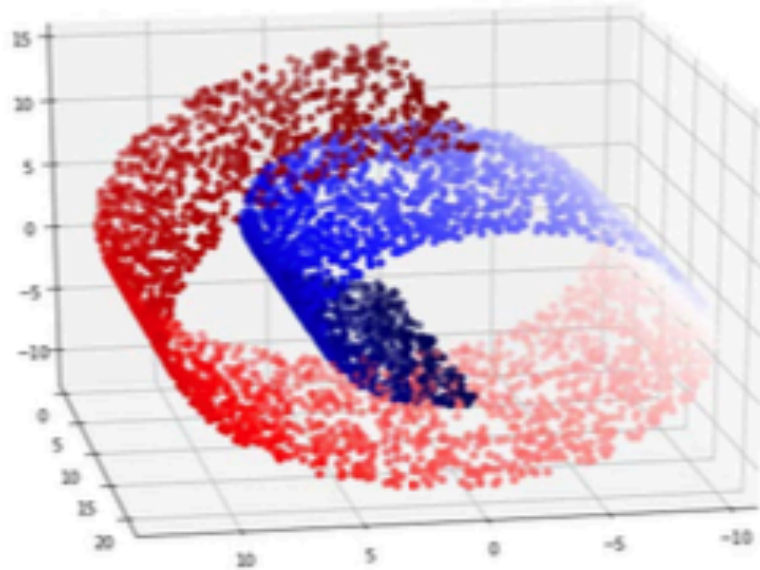
Drawback with global properties

- Global properties on some manifolds cannot characterize the manifold well

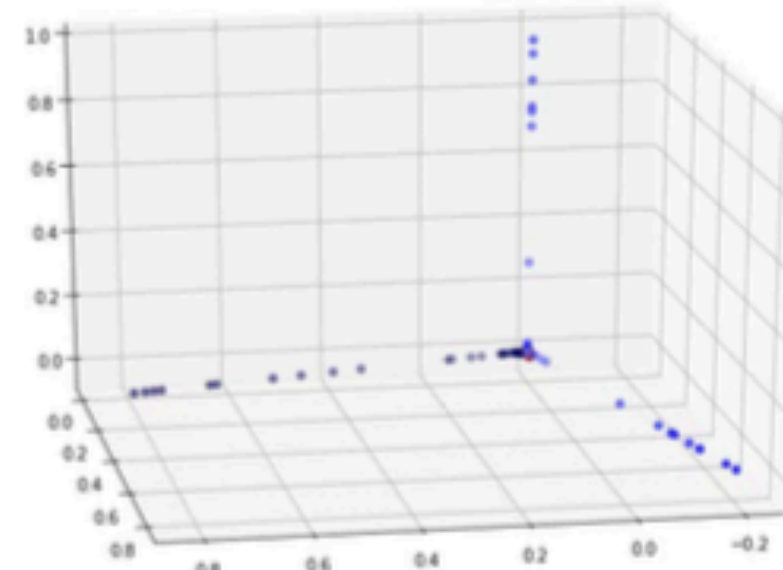


- Techniques based on preserving global properties does poorly on the Swiss roll

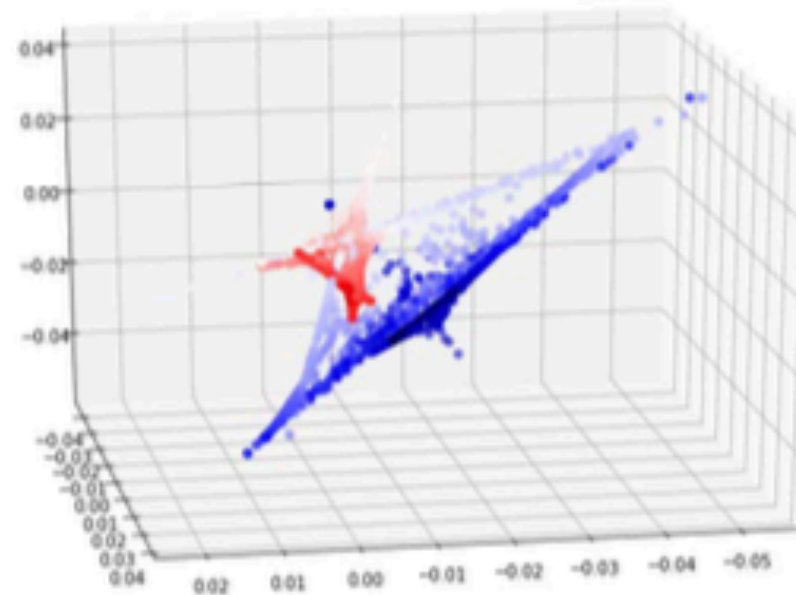
Drawback with global properties



Original Data

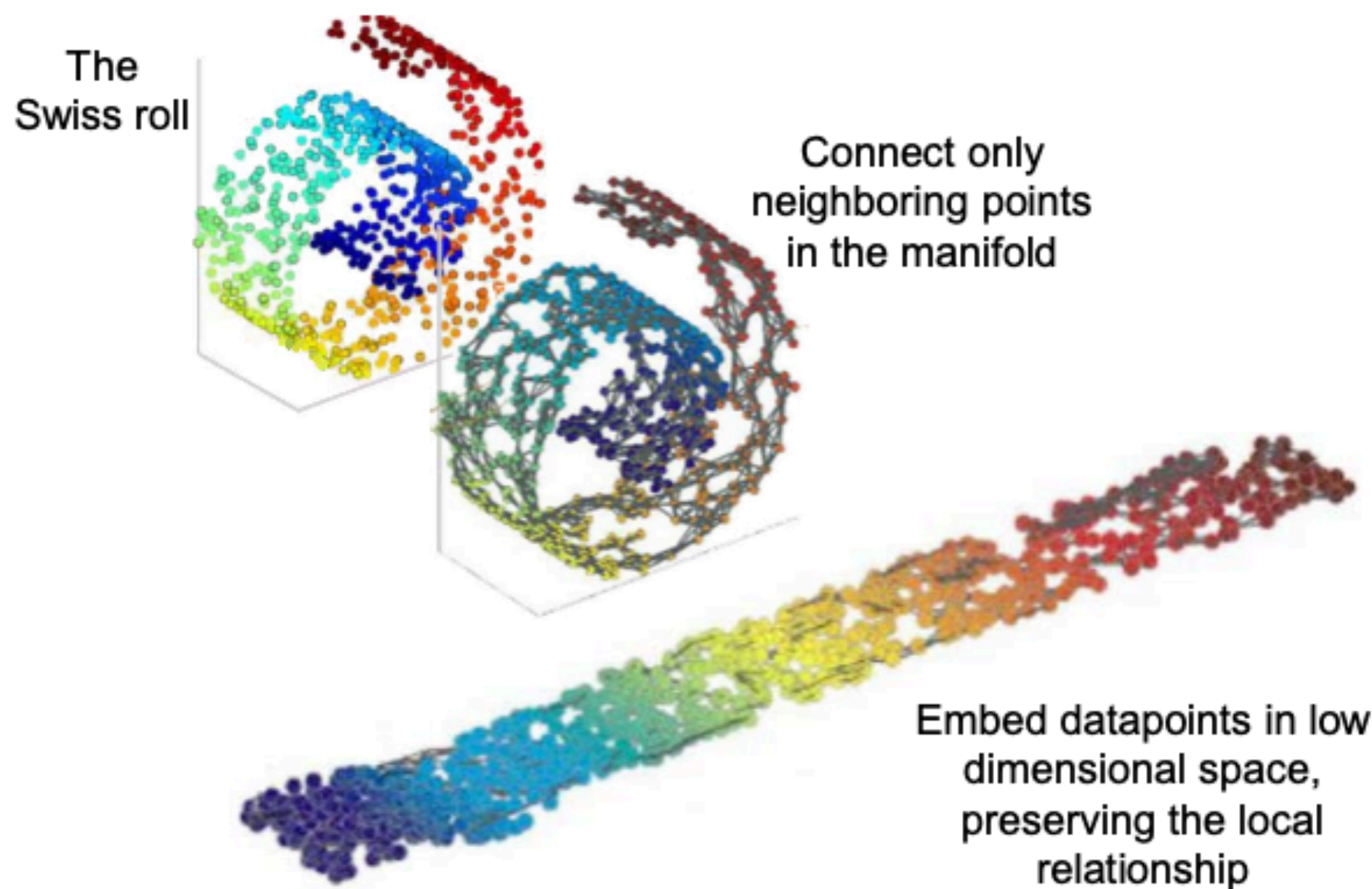


Data projected to main
three eigenvectors in
kernel PCA (rbf kernel)

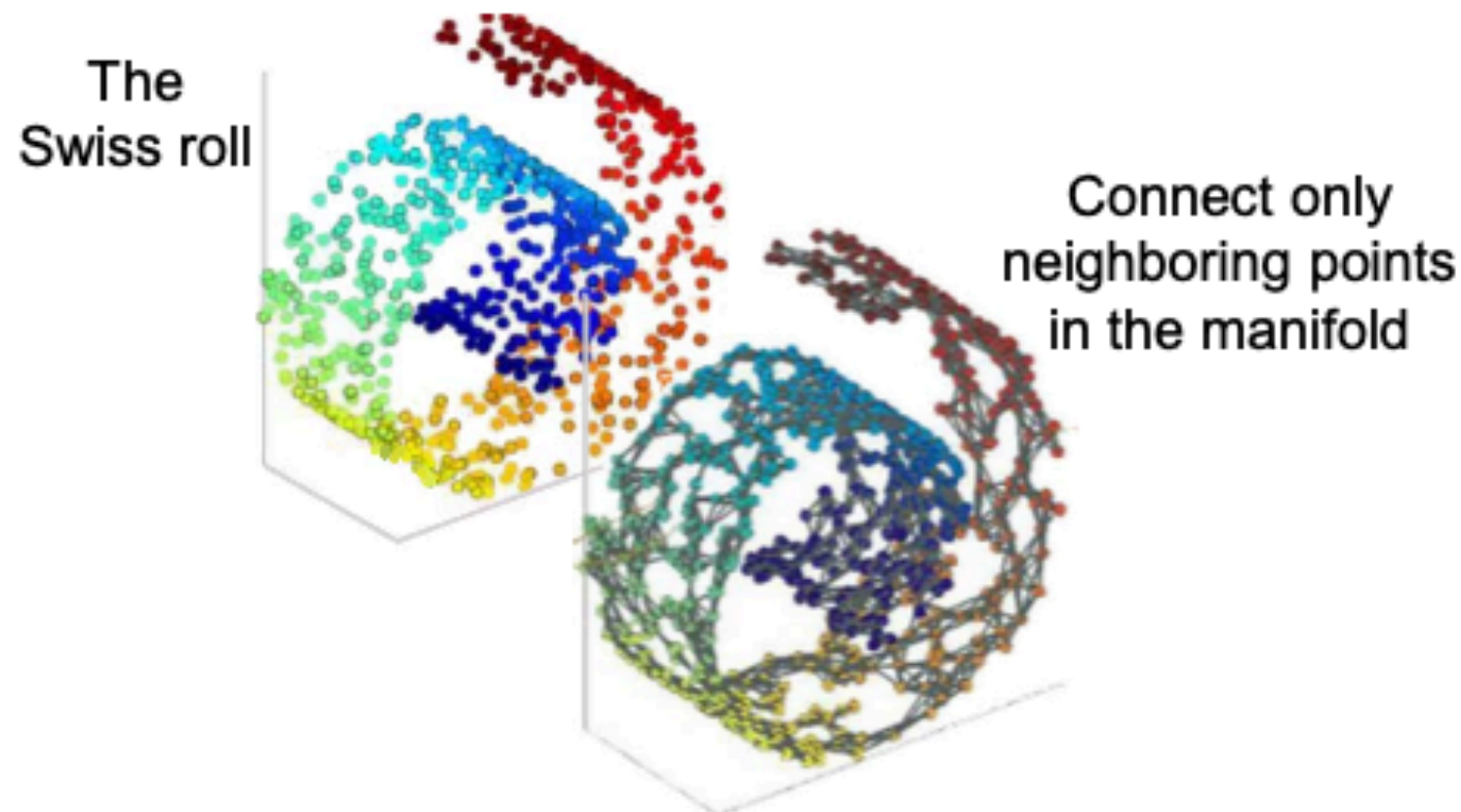


On the other hand,
methods such as **LLE**
that preserve local
properties can handle the
Swiss roll

Local structure preserving mapping



Isomap idea



- Isomap performs only the first step to find (Euclidean) distances of neighboring points
- Pairwise (geodesic) distances are estimated using the neighboring distances
- Then, MDS is used on the estimated geodesic distances

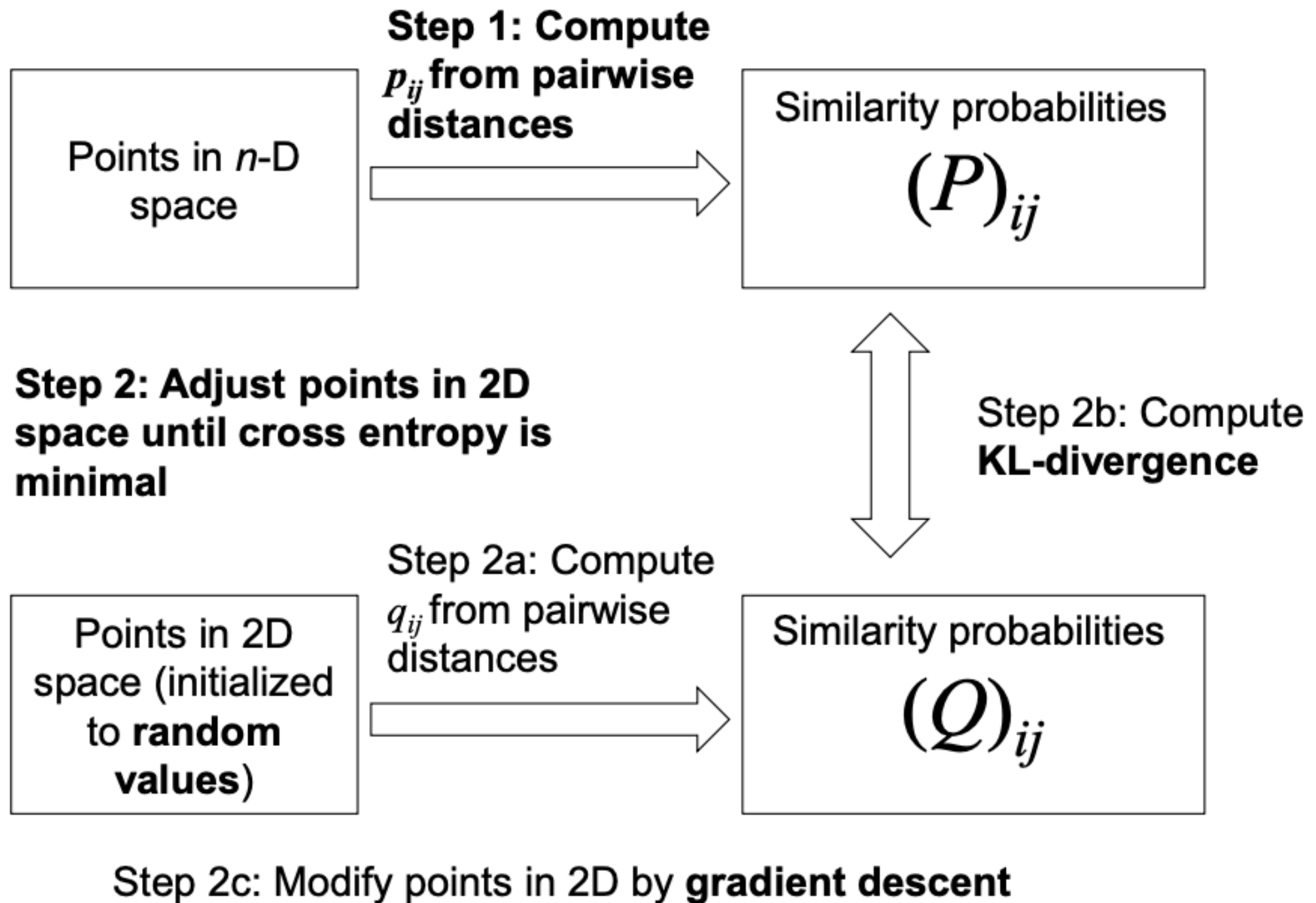
Isomap algorithm

1. Construct neighborhood graph
 - Find nearest k neighbors $N(x_i)$ of each point x_i
 - Construct a neighborhood graph by connecting x_i to the points in $N(x_i)$ with **Euclidean distance** set as edge weight
2. Compute (shortest) distance matrix M
 - Find shortest distance between pairwise points on the graph
3. Find eigenvectors of M using MDS (or PCA)

t-SNE in a nutshell

- Transform n -D space points into 2D
- Convert the distances in n -D space between pairwise points i, j into their **similarity probability** p_{ij} , assuming
 - Gaussian probability fall-off by the distance of the pair
 - Each point is at the mean of its own Gaussian with its own variance (determined through “perplexity”)
 - Result is matrix $(P)_{ij}$
- Reconstruct distances in 2D space (graph layout) such that they give rise to a similarity probability matrix $(Q)_{ij}$ with
 - t-distribution probability fall-off by the distance of each pair in the reconstructed points
 - The t-distributions all have the same variance (making clusters evenly sized in the 2D space for better visualization)
 - Minimal KL-divergence from $(P)_{ij}$

t-SNE in a nutshell

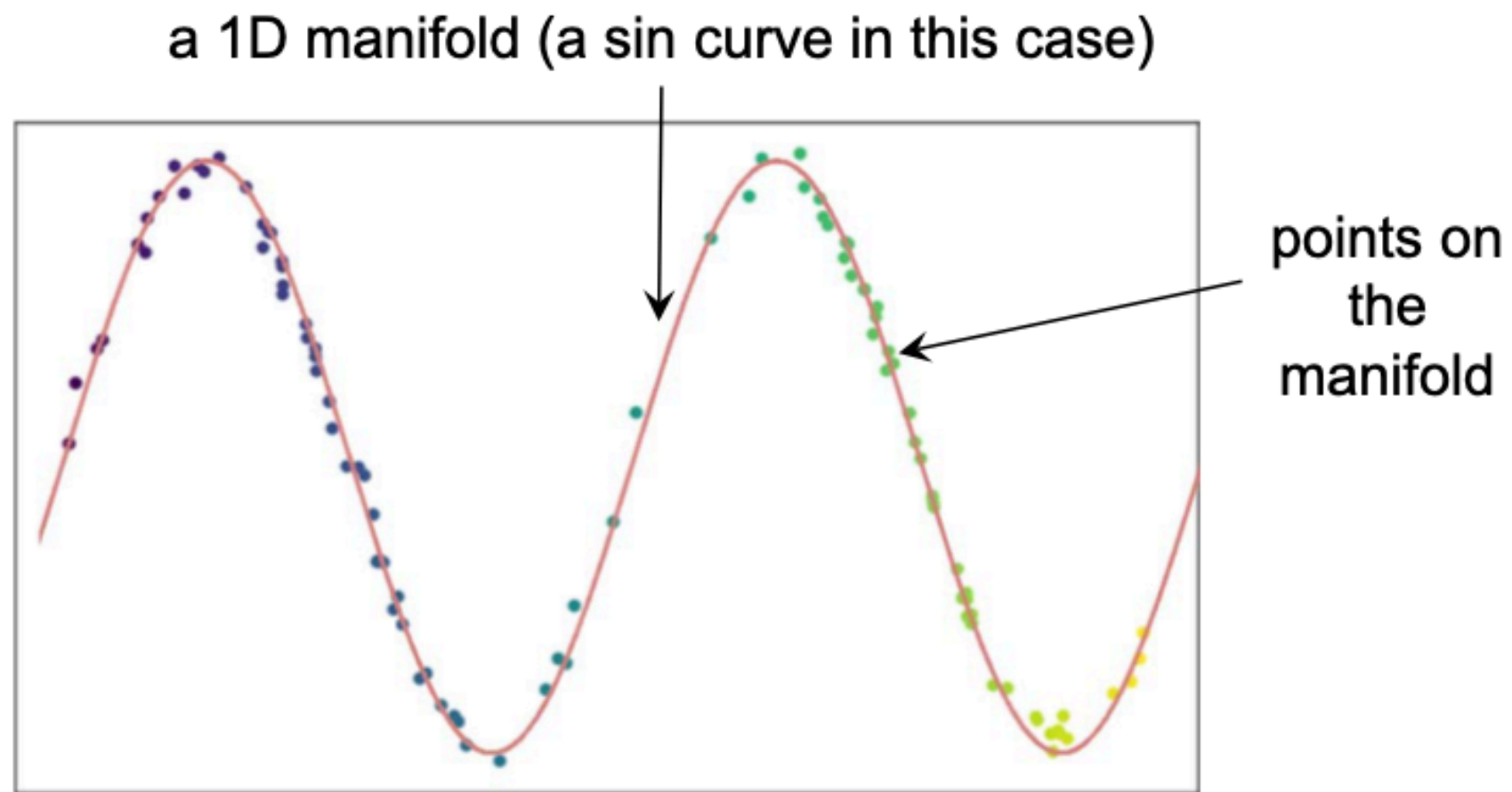


t-SNE lesson learned

- Like in Laplacian Eigenmap,
 $\exp\left(-\|x_i - x_j\|^2 / 2\sigma_i^2\right)$ is used for edge weight
 - However, t-SNE sets the edge weight for pairwise points, and count on natural elimination of edges between distant points
- Distance is adjusted for local data density through Perplexity
 - However the justification for Perplexity is weak
 - This is solved in UMAP

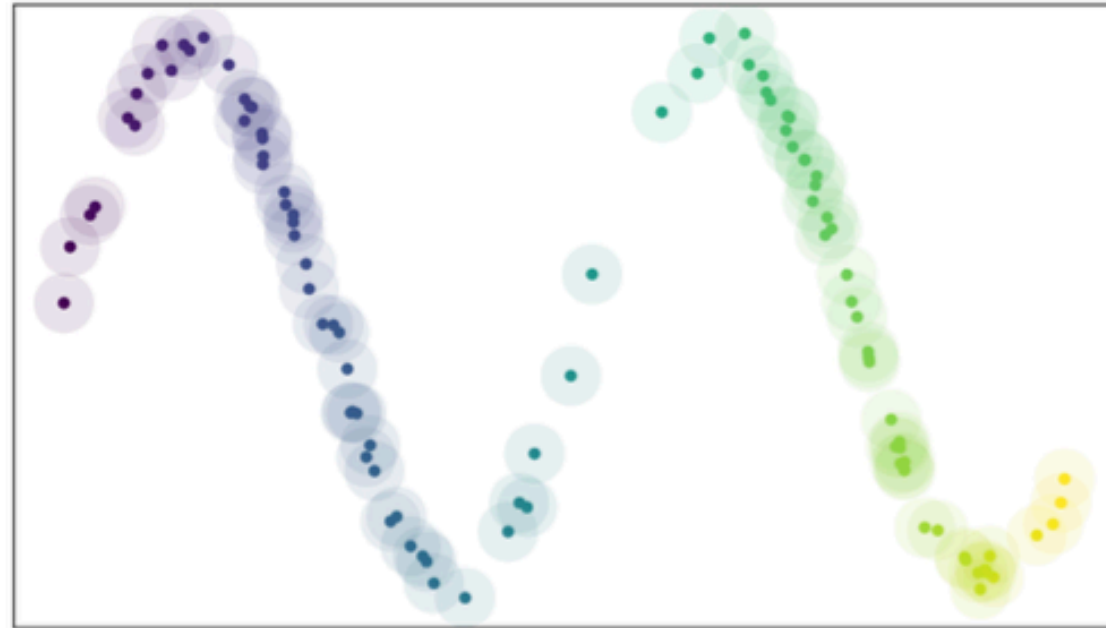
UMAP idea

- t-SNE converts the distance between two points into a probability in an *ad hoc* manner and attempts to preserve this probability
- UMAP starts with a totally different idea: To construct a manifold from only a sampling of the points on it

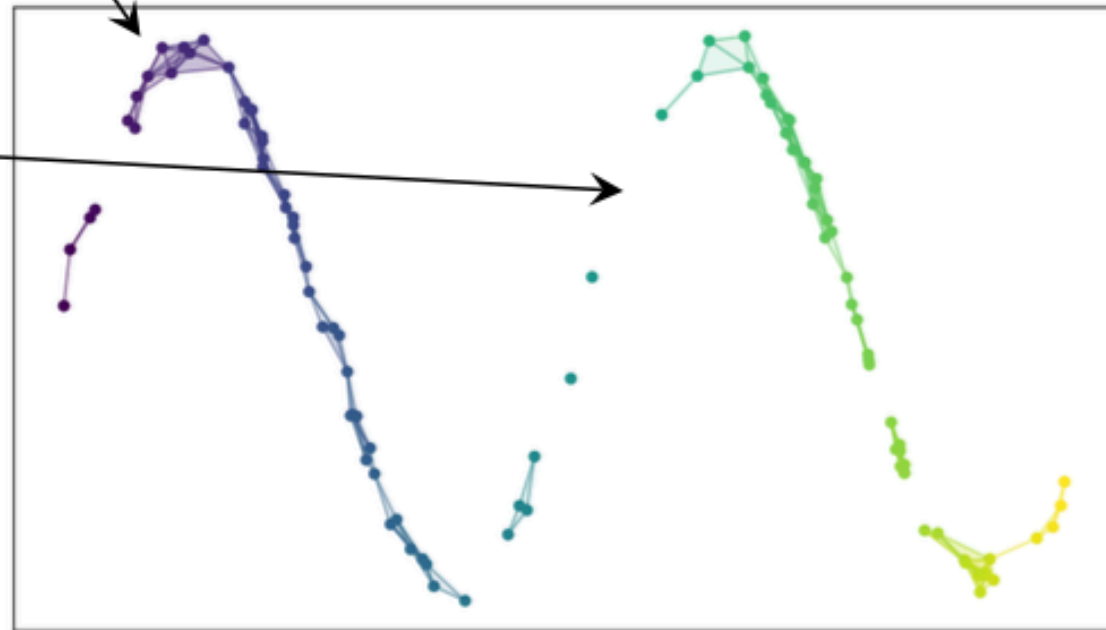


UMAP idea

- We can connect points that are of a fixed distance apart to build the objects that will allow us to recover the manifold



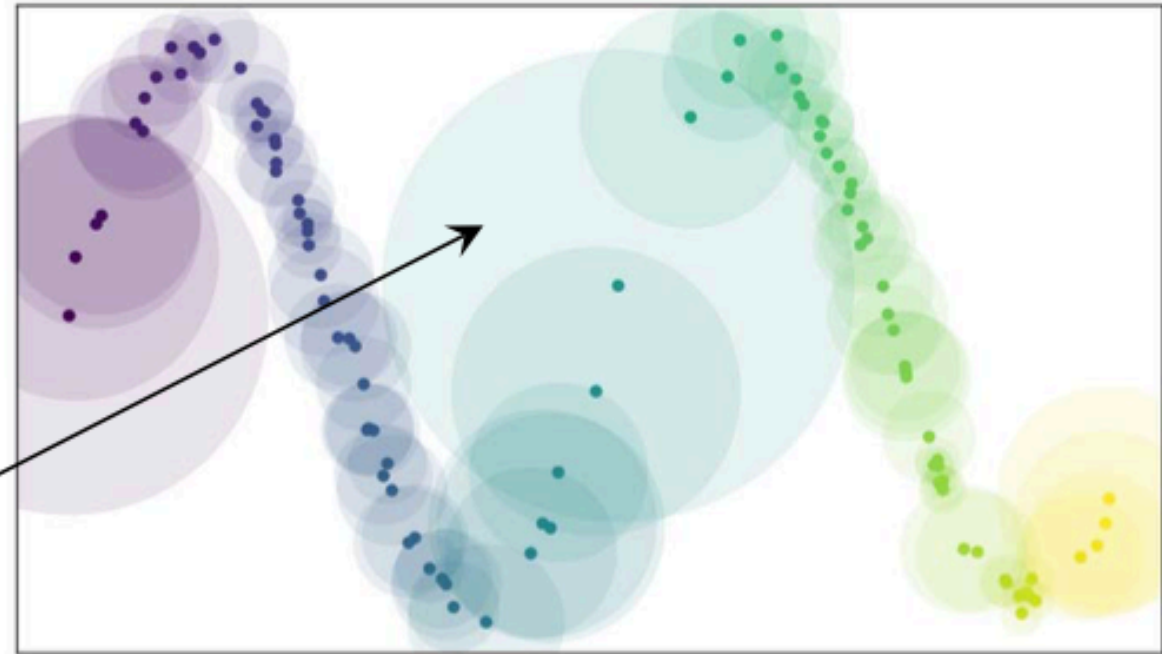
- But that will result in a lot of **missing regions** in the manifold



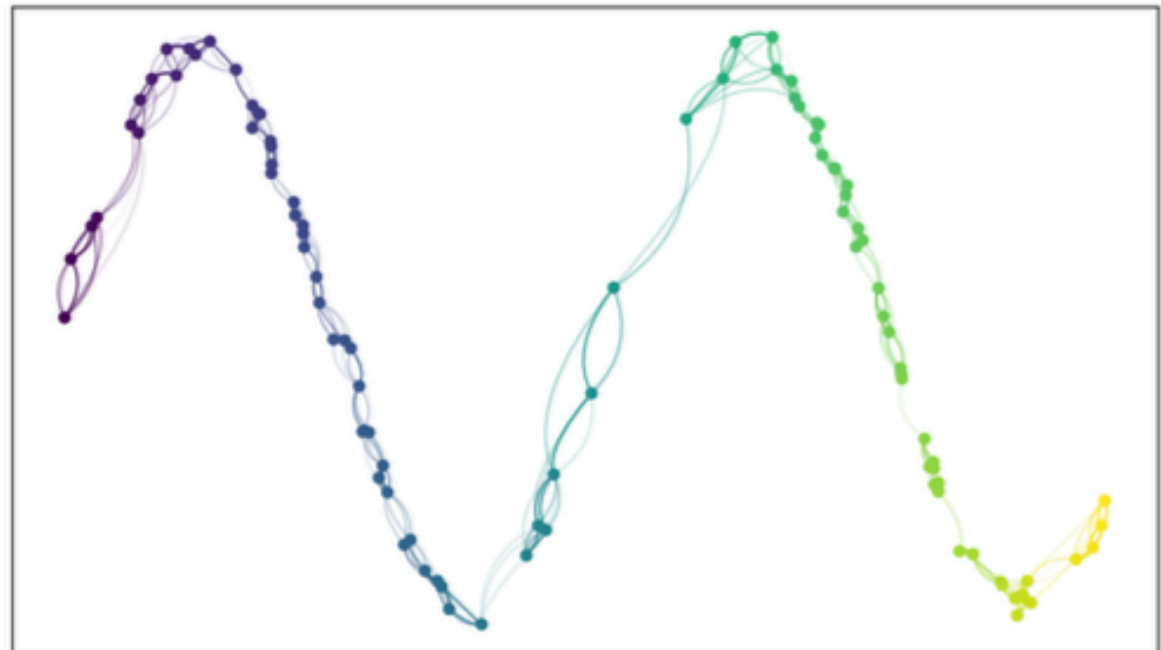
UMAP idea

- This can be solved if the distances are not treated as equal on the manifold

Give more weight
to sparse region



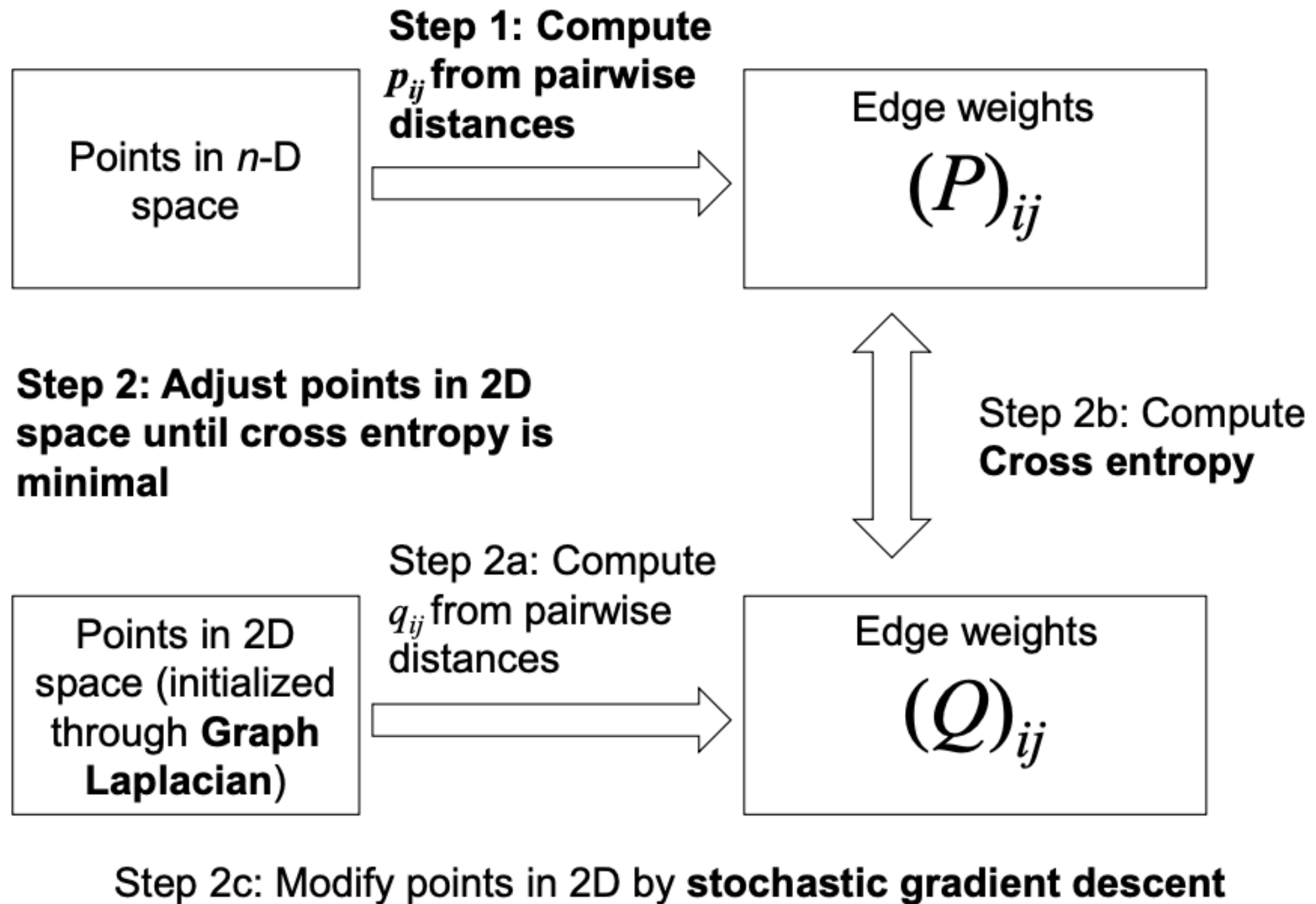
- The resultant objects will be able to cover the manifold



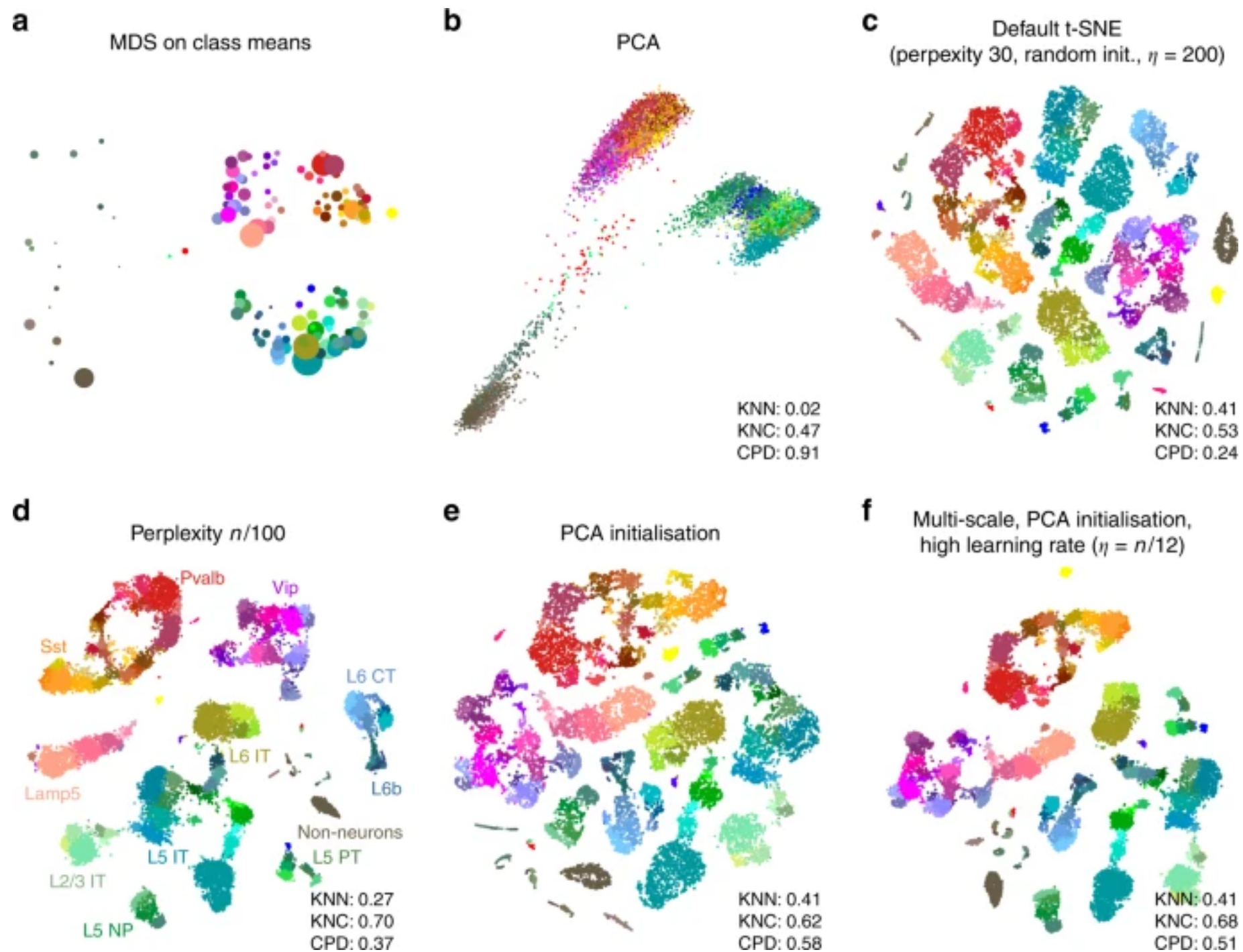
UMAP in a nutshell

- Convert the distances in n -D space between pairwise points i, j into an **edge weight** p_{ij} , assuming
 - Gaussian edge weight fall-off by distance
 - A Gaussian is assumed for each point, with variance determined through a **nearest-neighbor technique**
 - Result is matrix $(P)_{ij}$
- Reconstruct distances in 2D space (graph layout) such that they give rise to a matrix $(Q)_{ij}$ with
 - An **adjustable, t-distribution-like, edge weight fall-off** by the reconstructed distance
 - The distributions have same variance (making clusters evenly sized in the 2D space for better visualization)
 - Minimal **cross entropy** from $(P)_{ij}$

UMAP in a nutshell



tSNE applied to scRNA-seq

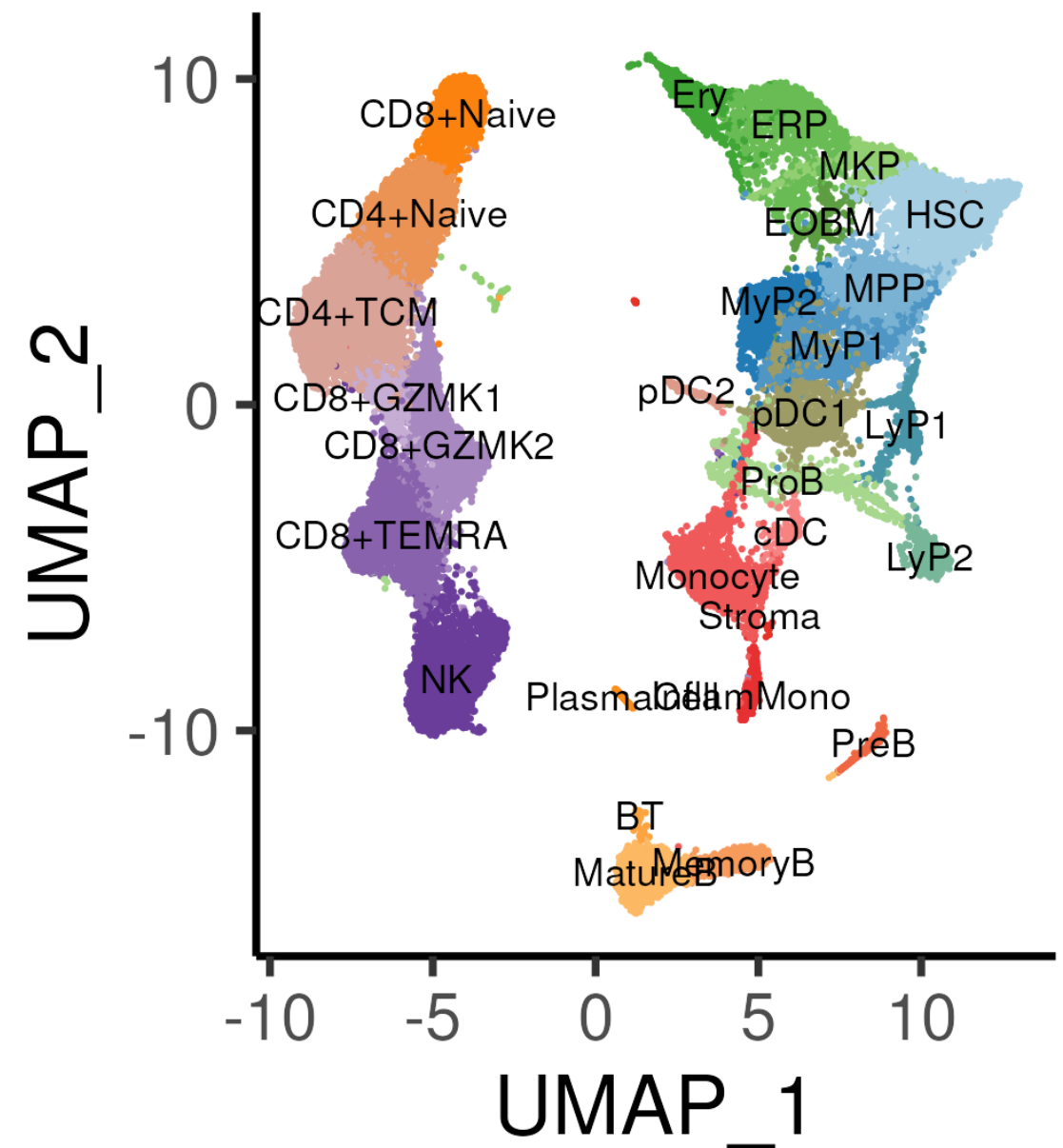
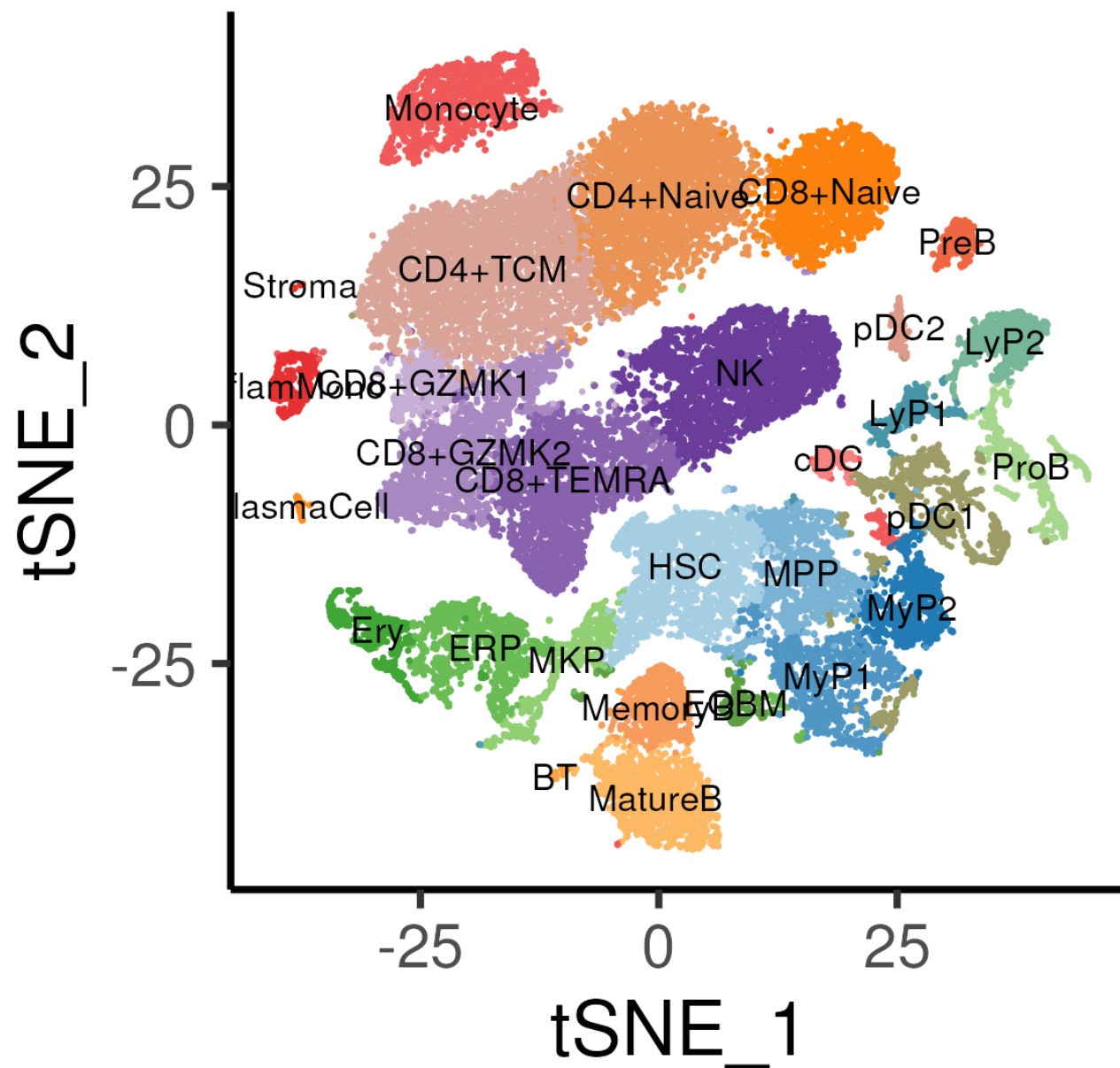


The art of using t-SNE for single-cell transcriptomics

Tasic et al. data set. Sample size = 23,822. Cluster assignments and cluster colours are taken from the original publication³. Warm colours correspond to inhibitory neurons, cold colours correspond to excitatory neurons, brown/grey colours correspond to non-neural cells.

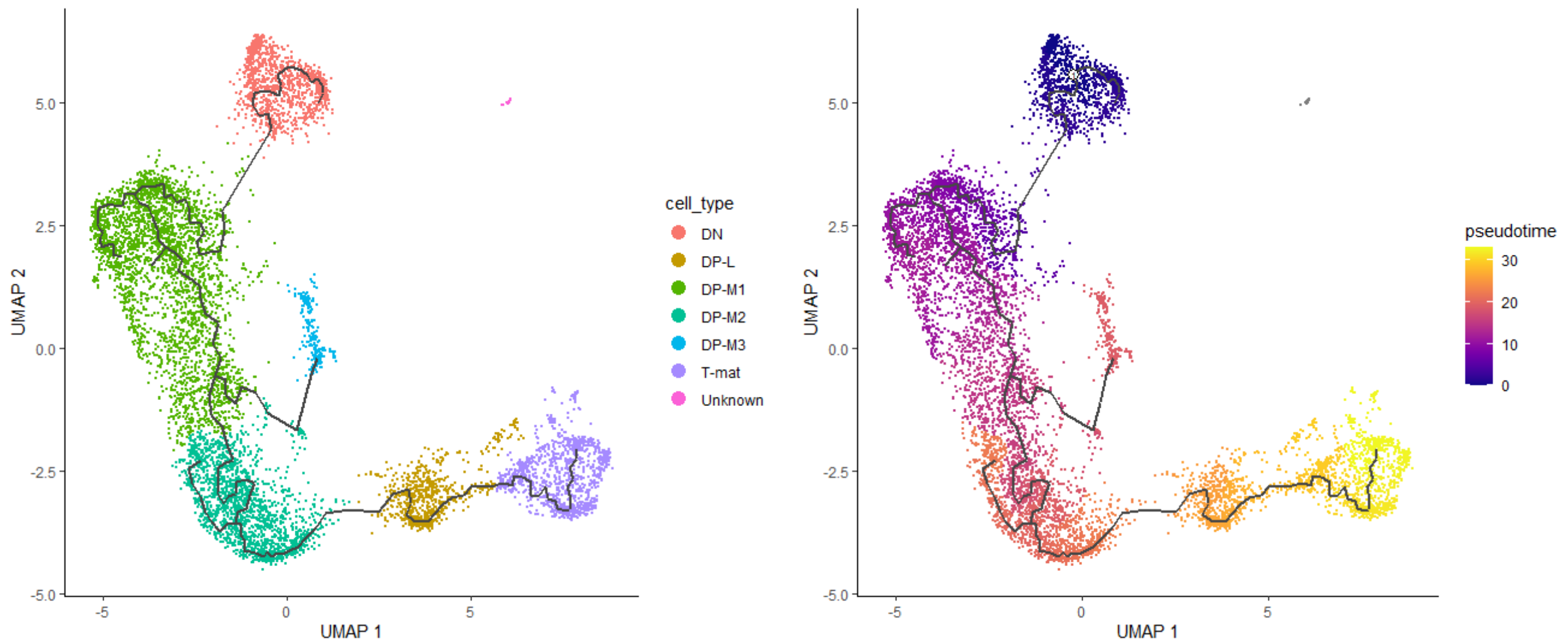
<https://www.nature.com/articles/s41467-019-13056-x>

tSNE & UMAP applied to scRNA-seq



t-SNE and UMAP projections of the bone marrow data coloured by cell annotation derived from previous section.

tSNE & UMAP applied to scRNA-seq



Trajectory inferred by Monocle 3

Single cell RNA-seq data from a mouse model of fetal growth restriction (Bacon *et al.* 2018).

https://training.galaxyproject.org/training-material/topics/single-cell/tutorials/scrna-case_monocle3-rstudio/tutorial.html

3D reconstruction from single cell RNA-seq

CSOmap: Reconstruction of cell spatial organization from single-cell RNA sequencing data based on ligand-receptor mediated self-assembly

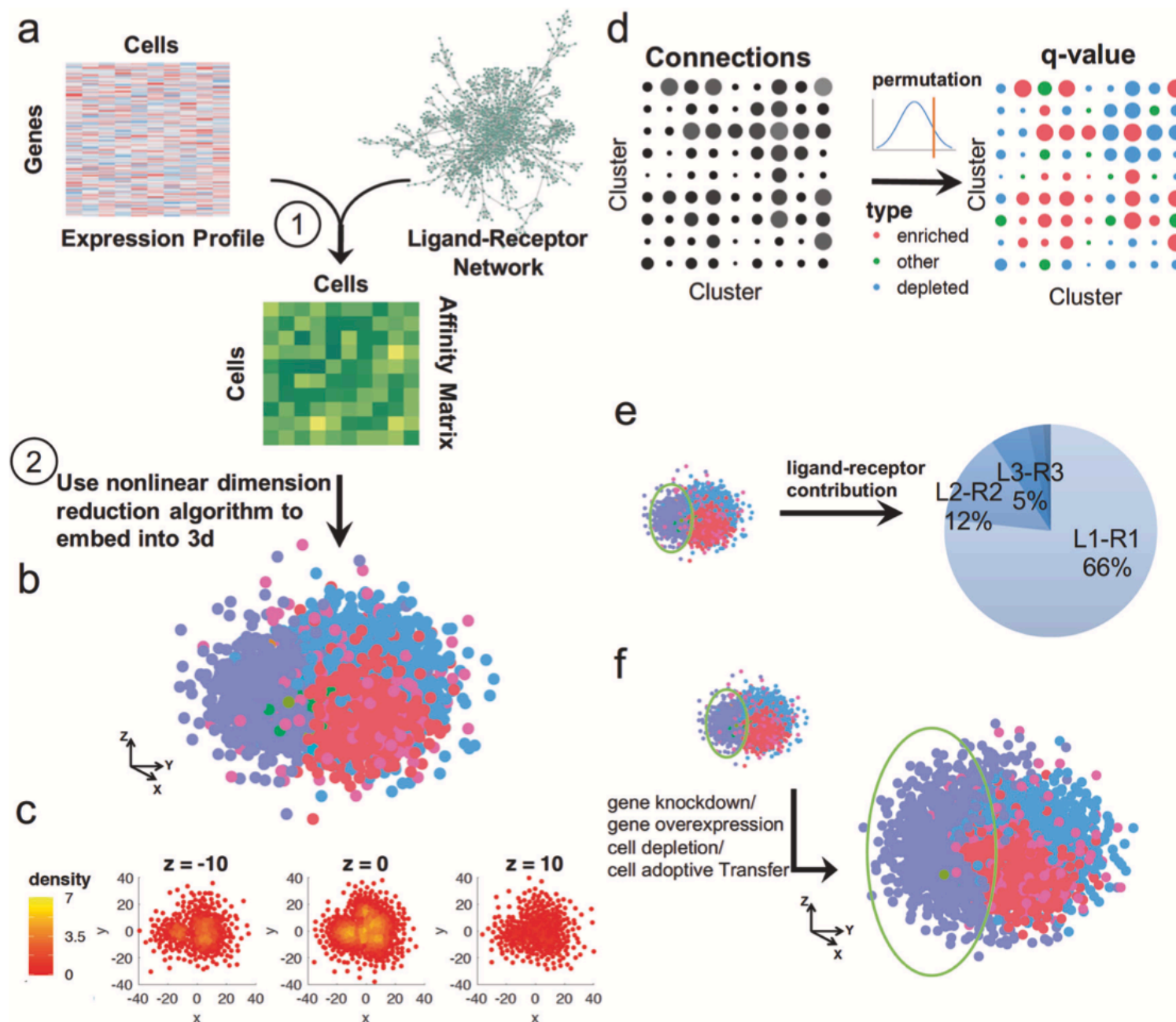


Fig. 1 Schematics of single-cell spatial reconstruction by CSOmap. **a** CSOmap takes the gene-by-cell expression matrix generated by scRNA-seq and the known ligand-receptor network as inputs, upon which a cell-by-cell affinity matrix is estimated. **b** The inherently high-dimensional cell-by-cell affinity matrix is embedded into a three-dimensional space via resolving cell competitions. **c** Density can be estimated for individual cells based on their three-dimensional coordinates obtained from **b**, which allows the identification of spatially-defined cell clusters. **d** Given the cell cluster labels, the number of connections among cell clusters and their statistical significance can be summarized and evaluated by CSOmap. **e** For a pair of cell clusters, the contributions of each ligand-receptor pair to their interactions can be calculated. **f** CSOmap allows in silico interference of the original dataset including gene knockdown/overexpression and cell depletion/adoptive transfer to examine the corresponding effects on cellular spatial organizations.

Overview of CSOmap

CSOmap reconstructs cellular spatial organization of individual cells from scRNA-seq data based on three principles: (1) cellular spatial organization is determined by ligand-receptor mediated cellular self-assembly, with cells having high affinity spatially close to each other; (2) the affinity of cells can be defined by the abundance of ligands and receptors and their interacting potentials; (3) **cells compete with each other to form spatial structures**. Hence, the core algorithm of CSOmap to reconstruct spatial organization of single cells from RNA-seq data includes two steps: (1) **estimating the cellular affinity matrix** based on the gene expression profiles of individual cells and known ligand-receptor interactions; (2) **embedding the inherently high-dimensional cellular affinity matrix into three-dimensional pseudo-space** resembling the realistic biological tissues during which cell competitions are sufficiently considered. CSOmap also includes additional algorithms for analyzing the resultant three-dimensional coordinates of single cells, including estimating the density of each cell, identifying spatially-defined cell clusters/ structures, evaluating the number of connections and statistical significance between two cell clusters defined by expression profiles or other characteristics, calculating the contributions of each ligand-receptor pair to the interaction potential of two cell clusters, and in silico molecular and cellular interference. The details of the core and additional algorithms of CSOmap are depicted as follows separately.

Embedding the high-dimensional cell-cell affinity matrix into three-dimensional space

When the discretized cell-by-cell affinity matrix is obtained, this inherently high-dimensional matrix is further embedded into a three-dimensional space. The principle behind this operation is that the realistic biological space is of only three dimensions and that positive affinity values in the affinity matrix only indicate the interacting potentials rather than realistically occurred facts. Cells having interacting potentials with common targets need to compete the space to change potentials to reality. Considering this factor, we introduce three constraints to build the computational model. First, a minimum distance between cells is pre-defined because all cells have positive sizes and cannot be ultimately squeezed. Second, the total available space is also pre-defined by a parameter named as space radius to simulate the limited realistic space. Finally and most importantly, Student's t-distribution is introduced to resolve the crowding issues of cell-cell interactions, **motivated by the visualization algorithm t-SNE, which allows cells to compete with others to form the spatial organization**. Taken all these considerations together, we propose the following computational model as the core of CSOmap:

$$\min \sum_{i=1}^n \sum_{j \neq i} p_{ij} \log \frac{p_{ij}}{q_{ij}} \quad (5)$$

subject to:

$$p_{ij} = \frac{1}{Z} \sum_{k=1}^K w_{L_k, R_k} (e_i^{L_k} \times e_j^{R_k} + e_i^{R_k} \times e_j^{L_k}) \text{ for } i \neq j \quad (6)$$

$$q_{ij} = \frac{1}{Z} \frac{1}{1 + d_{ij}^2} \text{ for } i \neq j \quad (7)$$

$$d_{ij}^2 = \sqrt{\sum_{k=1}^3 (y_i^k - y_j^k)^2} \text{ for } i \neq j \quad (8)$$

$$d_{ij} \geq r \text{ for } i \neq j \quad (9)$$

$$|y_i^k| \leq R \text{ for } i = 1 \dots n \text{ and } k = 1, 2, 3 \quad (10)$$



THANK YOU